

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ  
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ  
імені ІГОРЯ СІКОРСЬКОГО»**

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

До захисту допущено:  
**В.о. завідувача кафедри**  
**Михайло НОВОТАРСЬКИЙ**

(підпис)

«\_\_» \_\_\_\_\_ 2025 р.

**Дипломний проєкт**

**на здобуття ступеня бакалавра**

**за освітньо-професійною програмою «Інженерія програмного забезпечення  
комп'ютерних систем»**

**спеціальності 121 «Інженерія програмного забезпечення»**

на тему: Метод та програмні засоби побудови генераторів псевдовипадкових  
двійкових послідовностей для криптографічних застосувань

Виконав: студент \_\_\_\_\_ 4 \_\_\_\_\_ курсу, групи ІМ-11

\_\_\_\_\_  
Наливайко Ілля Ігорович

(прізвище, ім'я, по-батькові)

\_\_\_\_\_  
(підпис)

Керівник \_\_\_\_\_  
доцент, к.т.н. Марковський О. П.

(посада, науковий ступінь, прізвище та ініціали)

\_\_\_\_\_  
(підпис)

Консультант (нормоконтроль) старший викладач, доктор філософії  
комп'ютерної інженерії Міщенко Л. Д.

(назва розділу)

(посада, науковий ступінь, прізвище та ініціали)

\_\_\_\_\_  
(підпис)

Рецензент декан ФПМ, д.т.н проф. Дичка І. А.

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

\_\_\_\_\_  
(підпис)

Засвідчую, що у цьому дипломному  
проєкті немає запозичень з праць інших  
авторів без відповідних посилань.

Студент \_\_\_\_\_  
(підпис)

Київ – 2025 р.

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ  
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ  
імені ІГОРЯ СІКОРСЬКОГО»**

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Рівень вищої освіти – перший (бакалавр)

Освітньо-професійна програма

«Інженерія програмного забезпечення комп'ютерних систем»

Спеціальності 121 «Інженерія програмного забезпечення»

**ЗАТВЕРДЖУЮ:**

**В.о. завідувача кафедри**  
**Михайло НОВОТАРСЬКИЙ**

(підпис)

«\_\_» \_\_\_\_\_ 2025 р.

## **ЗАВДАННЯ**

на бакалаврський дипломний проєкт студента

Наливайка Іллі Ігоровича

1. Тема проєкту Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань  
керівник проєкту Марковський Олександр Петрович, доцент, к.т.н.  
затверджені наказом по університету від 23 травня 2025 року № 1705-с
2. Термін здачі студентом закінченого проєкту 3 червня 2025 р.
3. Вихідні дані до проєкту технічна документація, теоретичні дані
4. Зміст розрахунково-пояснювальної записки (перелік питань, які розробляються)

Розділ 1 ОГЛЯД ТА АНАЛІЗ СУЧАСНИХ ГЕНЕРАТОРІВ ПВП

Розділ 2 ТЕОРЕТИЧНЕ ДОСЛІДЖЕННЯ ВЛАСТИВОСТЕЙ

НЕЛІНІЙНИХ ФУНКЦІЙ ЗВОРОТНЬОГО ЗВ'ЯЗКУ ДЛЯ ЗСУВНОГО РЕГІСТРУ, ЩО ЗАБЕЗПЕЧУЮТЬ ПОВНИЙ ПЕРІОД

Розділ 3 РОЗРОБКА МЕТОДІВ СИНТЕЗУ ГПВДП НА ЗСУВНИХ РЕГІСТРАХ З НЕЛІНІЙНИМ ЗВОРОТНІМ ЗВ'ЯЗКОМ

Розділ 4 РОЗРОБКА ТА АНАЛІЗ ПРОГРАМНИХ ЗАСОБІВ ДЛЯ ПОБУДОВИ ГПВДП НА ЗСУВНИХ РЕГІСТРАХ З НЕЛІНІЙНИМ ЗВОРОТНІМ ЗВ'ЯЗКОМ

5. Перелік графічного матеріалу (з точним зображенням обов'язкових креслень): структура програмно-апаратного комплексу для генерації ПВП (структурна схема), блок-схема розширеного алгоритму (принципова схема), діаграма функціональних вимог (функціональна схема)
6. Консультанта проєкту, з вказівкою розділів проєкту, які до них вносяться

| Розділ        | Консультант   | Підпис, дата   |                  |
|---------------|---------------|----------------|------------------|
|               |               | Завдання видав | Завдання прийняв |
| Нормоконтроль | Міщенко Л. Д. |                |                  |
|               |               |                |                  |

7. Дата видачі завдання 10 січня 2025 р.  
Календарний план

| №  | Найменування етапів дипломного проєкту              | Терміни виконання етапів проєкту | Примітки |
|----|-----------------------------------------------------|----------------------------------|----------|
| 1. | Затвердження теми проєкту                           | 10.12.2024 - 15.01.2025          |          |
| 2. | Вивчення та аналіз завдання                         | 20.01.2025 - 10.02.2025          |          |
| 3. | Розробка архітектури та загальної структури системи | 10.02.2025 - 23.02.2025          |          |
| 4. | Розробка структур окремих підсистем                 | 23.02.2025 - 07.03.2025          |          |
| 5. | Програмна реалізація системи                        | 02.03.2025 - 03.05.2025          |          |
| 6. | Оформлення пояснювальної записки                    | 04.05.2025 - 02.06.2025          |          |
| 7. | Захист програмного продукту                         | 02.06.2025                       |          |
| 8. | Передзахист                                         | 04.06.2025                       |          |
| 9. | Захист                                              | 17.06.2025                       |          |

Студент

\_\_\_\_\_  
(підпис)

Ілля НАЛИВАЙКО

Керівник проєкту

\_\_\_\_\_  
(підпис)

Олександр МАРКОВСЬКИЙ

## АНОТАЦІЯ

У дипломній роботі розглядаються питання розробки та аналізу методів побудови генераторів псевдовипадкових двійкових послідовностей. Проаналізовано існуючі класи таких генераторів та серед них обрано генераторів на основі нелінійних регістрів зсуву як найбільш перспективних.

Запропоновано нові базову та розширені процедури синтезу нелінійних функцій зворотного зв'язку, що гарантують максимальний період генерації  $2^n$ . Наведено теоретичне обґрунтування властивостей таких функцій, включаючи умови для забезпечення повного циклу та аналіз їхньої кількості.

Для перевірки та імплементації запропонованих методів було створено програмний інструментарій, що базується на платформі Node.js. Програмний комплекс забезпечує перевірку функцій на повноту циклу, реалізацію базового та двох розширених алгоритмів синтезу з можливістю покрокового спостереження за процесом. Проведено аналіз обчислювальної складності реалізованих алгоритмів та для розширеного алгоритму обраховано результуючу кількість функцій повного циклу для деяких значень  $n$  довжини регістру.

Ключові слова: генератори псевдовипадкових двійкових послідовностей, криптографія, Node.js, нелінійні регістри зсуву, функції зворотного зв'язку, максимальний період.

## ANNOTATION

In this diploma thesis, the development and analysis of methods for constructing pseudorandom binary sequence generators is discussed. Existing classes of such generators have been analyzed, and among them, those based on nonlinear shift registers were identified as the most promising.

New basic and extended procedures for synthesizing nonlinear feedback functions that guarantee a maximum generation period of  $2^n$  are proposed. A theoretical justification of the properties of these functions is provided, including conditions for ensuring a full cycle and an analysis of their quantity.

To verify and implement the proposed methods, a software toolkit based on the Node.js platform was developed. This software package facilitates the testing of functions for full cycle properties and implements the basic and two extended synthesis algorithms, offering the possibility of step-by-step observation of the process. Additionally, a computational complexity analysis of the implemented algorithms was conducted, and for the extended algorithm, the resulting number of full-cycle functions was calculated for certain register lengths ( $n$ ).

**Keywords:** pseudorandom binary sequence generators, cryptography, Node.js, nonlinear shift registers, feedback functions, maximum period.

| Довідки   | Формат           | Значення           |        |      | Найменування                                                                                                                             | Кількість<br>литів                     | №<br>елементів | Додаток |
|-----------|------------------|--------------------|--------|------|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|----------------|---------|
|           |                  |                    |        |      | Документація загальна                                                                                                                    |                                        |                |         |
|           |                  |                    |        |      | Знову розроблена                                                                                                                         |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
|           | A4               | ІАЛЦ.467200.002 ТЗ |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань                 |                                        | 3              |         |
|           |                  |                    |        |      | Технічне завдання                                                                                                                        |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
|           | A4               | ІАЛЦ.467200.003 ПЗ |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань                 |                                        | 60             |         |
|           |                  |                    |        |      | Пояснювальна записка                                                                                                                     |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
|           | A4               | ІАЛЦ.467200.004 Д1 |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань                 |                                        | 1              |         |
|           |                  |                    |        |      | Структура програмно-апаратного комплексу для генерації ПВП (структурна схема)                                                            |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
|           | A4               | ІАЛЦ.467200.005 Д2 |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань                 |                                        | 1              |         |
|           |                  |                    |        |      | Блок-схема розширеного алгоритму                                                                                                         |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
|           | A4               | ІАЛЦ.467200.006 Д3 |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань                 |                                        | 1              |         |
|           |                  |                    |        |      | Діаграма функціональних вимог (функціональна схема)                                                                                      |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
|           | A4               | ІАЛЦ.467200.007 Д4 |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань                 |                                        | 12             |         |
|           |                  |                    |        |      | Текст програмного коду                                                                                                                   |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
|           |                  |                    |        |      | ІАЛЦ.467200.001 ОА                                                                                                                       |                                        |                |         |
|           |                  |                    |        |      |                                                                                                                                          |                                        |                |         |
| Зм.       | Арк.             | № докум.           | Підпис | Дата | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань<br>Опис альбому | Літера                                 | Аркуш          | Аркушів |
| Розробив  | Наливайко І. І.  |                    |        |      |                                                                                                                                          |                                        |                |         |
| Перевірив | Марковський О.П. |                    |        |      |                                                                                                                                          |                                        | 1              | 1       |
| Реценз.   | Дичка І. А.      |                    |        |      |                                                                                                                                          | КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11 |                |         |
| Н. контр. | Міщенко Л. Д.    |                    |        |      |                                                                                                                                          |                                        |                |         |
| Затверд.  |                  |                    |        |      |                                                                                                                                          |                                        |                |         |

**ТЕХНІЧНЕ ЗАВДАННЯ**  
**ДО ДИПЛОМНОГО ПРОЄКТУ**

на тему: «Метод та програмні засоби побудови генераторів псевдовипадкових  
двійкових послідовностей для криптографічних застосувань»

Київ – 2025 р.

# ЗМІСТ

|                                           |   |
|-------------------------------------------|---|
| НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ..... | 2 |
| ПІДСТАВИ ДЛЯ РОЗРОБКИ.....                | 2 |
| МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ.....         | 2 |
| ДЖЕРЕЛА РОЗРОБКИ.....                     | 2 |
| ТЕХНІЧНІ ВИМОГИ .....                     | 3 |
| Вимоги до розробленого продукту .....     | 3 |
| Вимоги до програмного забезпечення .....  | 3 |
| Вимоги до апаратної частини .....         | 3 |
| ЕТАПИ РОЗРОБКИ.....                       | 3 |

|           |  |                   |        |      |                                                                                                                                                         |                                               |       |         |  |
|-----------|--|-------------------|--------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|-------|---------|--|
|           |  |                   |        |      | ІАЛЦ.467200.002 ТЗ                                                                                                                                      |                                               |       |         |  |
|           |  | № докум.          | Підпис | Дата |                                                                                                                                                         |                                               |       |         |  |
| Розробив  |  | Наливайко І.І.    |        |      | Метод та програмні засоби побудови генераторів<br>псевдовипадкових двійкових послідовностей для<br>криптографічних застосувань<br><br>Технічне завдання | Літ.                                          | Аркуш | Аркушів |  |
| Перевірив |  | Марковський О. П. |        |      |                                                                                                                                                         |                                               | 1     | 3       |  |
| Реценз.   |  | Дичка І. А.       |        |      |                                                                                                                                                         | НТУУ КПІ ім. Ігоря<br>Сікорського, ФІОТ,ІМ-11 |       |         |  |
| Н. Контр. |  | Мищенко Л. М.     |        |      |                                                                                                                                                         |                                               |       |         |  |
| Затвердив |  |                   |        |      |                                                                                                                                                         |                                               |       |         |  |



# **1 НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ**

Технічне завдання поширюється на розробку програмного комплексу синтезу, перевірки та аналізу генераторів псевдовипадкових двійкових послідовностей на основі нелінійних функцій зворотного зв'язку.

Область застосування системи є формування криптографічних примітивів, включаючи елементи потокових шифрів та процедури генерації ключів.

## **2 ПІДСТАВИ ДЛЯ РОЗРОБКИ**

Підставою для розробки даної програмної компоненти є завдання на виконання бакалаврського проєкту за освітньо-професійною програмою «Інженерія програмного забезпечення комп'ютерних систем» спеціальності 121 «Інженерія програмного забезпечення», затверджене кафедрою обчислювальної техніки факультету інформатики та обчислювальної техніки Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

## **3 МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ**

Метою та призначенням цієї роботи є розробка методів побудови та програмних засобів синтезу криптографічно стійких генераторів псевдовипадкових двійкових послідовностей на основі нелінійних функцій зворотного зв'язку.

## **4 ДЖЕРЕЛА РОЗРОБКИ**

Джерелом розробки є наукові публікації та статті на дану тему, науково-технічна література, документації та публікації в мережі Інтернет, що є загальнодоступними.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.002 ТЗ | Арк. |
|     |      |          |        |      |                    | 2    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

## 5 ТЕХНІЧНІ ВИМОГИ

### 5.1. Вимоги до розробленого продукту

Розроблена система має виконувати наступні вимоги:

- Алгоритм повинен синтезувати нелінійну функцію зворотного зв'язку, що забезпечує повний цикл для генератора псевдовипадкових послідовностей.
- Обчислювальна складність методу синтезу функції зворотного зв'язку повинна бути нижчою за складність повного перебору усіх можливих функцій для заданої розрядності  $n$ .
- Реалізовані програмні засоби мають підтримувати покрокову візуалізацію процесу синтезу функції для цілей подальшого аналізу та верифікації.

### 5.2. Вимоги до програмного забезпечення

- ОС Windows, Linux або Mac.
- Visual Studio Code версії 1.60.0 або вище.
- Встановлений Node.js версії 16.x LTS або вище.

### 5.3. Вимоги до апаратної частини

- ЦП рівня не нижче, ніж Intel Core i3 8-го покоління.
- RAM не менше, ніж 4 ГБ.

## 6 ЕТАПИ РОЗРОБКИ

| №  | Назва етапів виконання                              | Терміни виконання етапів проєкту | Примітки |
|----|-----------------------------------------------------|----------------------------------|----------|
| 1. | Затвердження теми проєкту                           | 10.12.2024 - 15.01.2025          |          |
| 2. | Вивчення та аналіз завдання                         | 20.01.2025 - 10.02.2025          |          |
| 3. | Розробка архітектури та загальної структури системи | 10.02.2025 - 23.02.2025          |          |
| 4. | Розробка структур окремих підсистем                 | 23.02.2025 - 07.03.2025          |          |
| 5. | Програмна реалізація системи                        | 02.03.2025 - 03.05.2025          |          |
| 6. | Виправлення помилок                                 | 03.05.2025                       |          |
| 7. | Оформлення пояснювальної записки                    | 04.05.2025 - 02.06.2025          |          |

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.002 ТЗ | Арк. |
|     |      |          |        |      |                    | 3    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

**ПОЯСНЮВАЛЬНА ЗАПИСКА  
ДО ДИПЛОМНОГО ПРОЄКТУ**

на тему: *«Метод та програмні засоби побудови генераторів псевдовипадкових  
двійкових послідовностей для криптографічних застосувань»*

Київ – 2025 р.

# ЗМІСТ

|                                                                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|----|
| ПЕРЕЛІК СКОРОЧЕНЬ.....                                                                                                                              | 3  |
| ВСТУП .....                                                                                                                                         | 4  |
| РОЗДІЛ 1 ОГЛЯД ТА АНАЛІЗ СУЧАСНИХ ГЕНЕРАТОРІВ ПВП.....                                                                                              | 6  |
| 1.1. Варіанти застосування ПВП .....                                                                                                                | 6  |
| 1.2 Критерії ефективності та класифікація ГПВП.....                                                                                                 | 7  |
| 1.3 Огляд архітектури та алгоритмів існуючих ГПВП .....                                                                                             | 9  |
| 1.3.1 Лінійний конгруентний ГПВП .....                                                                                                              | 9  |
| 1.3.2 Степенний конгруентний ГПВП .....                                                                                                             | 10 |
| 1.3.3 Поліноміальний конгруентний ГПВП .....                                                                                                        | 11 |
| 1.3.4 ГПВП RSA .....                                                                                                                                | 11 |
| 1.3.5 ГПВП на регістрах зсуву з лінійними зворотними зв'язками .....                                                                                | 12 |
| 1.3.6 ГПВДП довільної довжини.....                                                                                                                  | 14 |
| 1.3.7 Мультиплексні ГПВП .....                                                                                                                      | 15 |
| 1.3.8 ГПВП Геффа.....                                                                                                                               | 15 |
| 1.3.9 Старт-стопний ГПВП .....                                                                                                                      | 16 |
| 1.4 Аналіз використання ГПВП у комп'ютерних системах .....                                                                                          | 17 |
| ВИСНОВОК ДО РОЗДІЛУ 1 .....                                                                                                                         | 18 |
| РОЗДІЛ 2 ТЕОРЕТИЧНЕ ДОСЛІДЖЕННЯ ВЛАСТИВОСТЕЙ НЕЛІНІЙНИХ<br>ФУНКЦІЙ ЗВОРОТНЬОГО ЗВ'ЯЗКУ ДЛЯ ЗСУВНОГО РЕГІСТРУ, ЩО<br>ЗАБЕЗПЕЧУЮТЬ ПОВНИЙ ПЕРІОД..... | 20 |
| 2.1 Загальні властивості нелінійних функцій зворотного зв'язку, що<br>забезпечують максимальний період.....                                         | 20 |
| 2.2 Аналіз кількості функцій зворотного зв'язку, які забезпечують<br>максимальний період .....                                                      | 26 |
| ВИСНОВОК ДО РОЗДІЛУ 2 .....                                                                                                                         | 28 |
| РОЗДІЛ 3 РОЗРОБКА МЕТОДІВ СИНТЕЗУ ГПВДП НА ЗСУВНИХ<br>РЕГІСТРАХ З НЕЛІНІЙНИМ ЗВОРОТНІМ ЗВ'ЯЗКОМ.....                                                | 29 |

|           |                  |          |        |      |                                                                                                                                                            |                                               |       |         |  |
|-----------|------------------|----------|--------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|-------|---------|--|
|           |                  |          |        |      | ІАЛЦ.467200.003 ПЗ                                                                                                                                         |                                               |       |         |  |
|           |                  |          |        |      |                                                                                                                                                            |                                               |       |         |  |
| Зм.       | Арк.             | № докум. | Підпис | Дата |                                                                                                                                                            |                                               |       |         |  |
| Розробив  | Наливайко І.І.   |          |        |      | Метод та програмні засоби побудови генераторів<br>псевдовипадкових двійкових послідовностей для<br>криптографічних застосувань<br><br>Пояснювальна записка | Літ.                                          | Аркуш | Аркушів |  |
| Перевірів | Марковський О.П. |          |        |      |                                                                                                                                                            |                                               | 1     | 60      |  |
| Реценз.   | Дичка І. А.      |          |        |      |                                                                                                                                                            | КПІ ім. Ігоря Сікорського,<br><br>ФІОТ, ІМ-11 |       |         |  |
| Н. Контр. | Міщенко Л. Д.    |          |        |      |                                                                                                                                                            |                                               |       |         |  |
| Затвердив |                  |          |        |      |                                                                                                                                                            |                                               |       |         |  |

|                                                                                                                              |    |
|------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 Базова процедура побудови NLFSR на основі об'єднання мінімальних елементів замкнутих кодових груп .....                  | 29 |
| 3.1.1 Алгоритм синтезу NLFSR.....                                                                                            | 29 |
| 3.1.2 Приклад роботи алгоритму для $n = 4$ .....                                                                             | 32 |
| 3.2 Розширена процедура побудови NLFSR на основі об'єднання мінімальних елементів замкнутих кодових груп .....               | 34 |
| 3.2.1 Алгоритм синтезу NLFSR.....                                                                                            | 34 |
| 3.2.2 Приклад роботи алгоритму для $n = 4$ .....                                                                             | 36 |
| ВИСНОВОК ДО РОЗДІЛУ 3 .....                                                                                                  | 39 |
| РОЗДІЛ 4 РОЗРОБКА ТА АНАЛІЗ ПРОГРАМНИХ ЗАСОБІВ ДЛЯ ПОБУДОВИ ГПВДП НА ЗСУВНИХ РЕГІСТРАХ З НЕЛІНІЙНИМ ЗВОРОТНІМ ЗВ'ЯЗКОМ ..... | 41 |
| 4.1 Загальна архітектура та структура програмного проєкту.....                                                               | 41 |
| 4.2 Реалізація алгоритмів синтезу NLFSR.....                                                                                 | 44 |
| 4.2.1 Реалізація базового алгоритму baseAlgorithm.js.....                                                                    | 44 |
| 4.2.2 Реалізація розширеного алгоритму в 2 режимах.....                                                                      | 46 |
| 4.3 Аналіз отриманих результатів та обчислювальної складності .....                                                          | 50 |
| ВИСНОВОК ДО РОЗДІЛУ 4 .....                                                                                                  | 54 |
| ВИСНОВКИ.....                                                                                                                | 56 |
| СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ .....                                                                                         | 58 |

## ПЕРЕЛІК СКОРОЧЕНЬ

|       |                                                                                                |
|-------|------------------------------------------------------------------------------------------------|
| ГВП   | Генератор випадкових послідовностей                                                            |
| ГПВП  | Генератор псевдовипадкових послідовностей                                                      |
| ГПВДП | Генератор псевдовипадкових двійкових послідовностей                                            |
| LFSR  | Linear Feedback Shift Register (зсувний регістр з лінійною функцією зворотнього зв'язку)       |
| NLFSR | Non-Linear Feedback Shift Register (зсувний регістр з нелінійною функцією зворотнього зв'язку) |

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 3    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

## ВСТУП

У сучасному інформаційному суспільстві, де критично важливо забезпечити збереження даних та стабільність цифрових комунікацій, безпека інформації набуває особливого значення. Зі зростанням обсягів інформації, що передається через відкриті канали, а також зі зростанням складності кіберзагроз, завдання розробки ефективних криптографічних засобів набуває першорядного значення для спеціалістів у галузі програмного забезпечення та комп'ютерної інженерії. Постійні виклики сучасної кіберсфери спонукають до пошуку інноваційних підходів для захисту даних, що сприяє подальшому розвитку дослідницької думки в цій сфері.

Одним із ключових елементів численних криптографічних систем є генератори псевдовипадкових послідовностей, від яких безпосередньо залежить стійкість та ефективність алгоритмів шифрування, протоколів автентифікації та інших механізмів захисту [1]. Якість таких генераторів визначає не лише рівень безпеки, але й стабільність криптографічних процесів, оскільки вони використовуються для формування ключів, генерації векторів ініціалізації, одноразових кодів і забезпечення загальної криптографічної надійності систем. Побудова генераторів, які одночасно відповідають високим вимогам щодо криптографічної стійкості, статистичної якості та практичної ефективності, являє собою складну науково-технічну задачу, вирішення якої вимагає комплексного підходу та глибокого аналізу [2].

Метою даного дипломного проєкту є дослідження і розробка методів побудови генераторів псевдовипадкових двійкових послідовностей, орієнтованих на криптографічні застосування. Проєкт також передбачає створення спеціалізованих програмних засобів для реалізації та аналізу запропонованих методів, що дозволить забезпечити перевірку їхньої ефективності та стабільності в реальних умовах експлуатації.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 4    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Всебічний огляд існуючих підходів до створення генераторів допоможе виявити як переваги, так і недоліки відомих рішень, зокрема тих, що базуються на лінійних та нелінійних регістрах зсуву. Такий аналіз створить можливість для визначення подальших напрямків удосконалення, розробки нових методів та оптимізації існуючих алгоритмів з урахуванням сучасних вимог до продуктивності і криптографічної стійкості.

Ця дипломна робота спрямована на створення оптимізованого алгоритму генерації псевдовипадкових послідовностей, який має потенціал значно підвищити надійність і безпеку сучасних інформаційних систем, відкриваючи нові можливості для практичного застосування ефективних криптографічних генераторів.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 5    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |



# РОЗДІЛ 1

## ОГЛЯД ТА АНАЛІЗ СУЧАСНИХ ГЕНЕРАТОРІВ ПВП

### 1.1. Варіанти застосування ПВП

ГВП – це пристрій чи алгоритм, який видає ряд незалежних і непередбачуваних двійкових розрядів. Щоб гарантувати дійсну випадковість його результатів, така система обов’язково повинна використовувати природне джерело невизначеності.

На відміну від ГВП, ГПВП працює за детермінованим алгоритмом. Він приймає деяку двійкову послідовність (англ. seed) довжиною  $k$  і генерує з неї послідовність бітів довжиною  $l$ , де  $l$  значно перевищує  $k$ . Хоча отриманий результат і має статистичні характеристики, близькі до випадкових, він залишається строго визначеним, якщо відтворити задані початкові значення [3].

Незважаючи на імітацію випадковості, вихідна послідовність ГПВП не є абсолютно випадковою. Її можливий набір формується лише з  $2^k$  варіантів, що становить лише невелику частину від  $2^l$  усіх потенційних комбінацій бітів заданої довжини.

Основна різниця полягає в тому, що при використанні однакового початкового значення (seed) ГПВП завжди видає ту саму послідовність, що свідчить про її детермінованість. Для генераторів випадкових послідовностей, щоб повторити той самий результат, потрібно задати початкові умови з абсолютною, нескінченною точністю – вимога, яка у більшості випадків є фізично недосяжною.

Зазвичай, кожне наступне значення в алгоритмі ГПВП обчислюється на основі попереднього [4]. Класична схема такого генератора, включає блок пам'яті для зберігання поточного вектора стану, який при ініціалізації заповнюється початковим кодом (що може виконувати роль ключа). До цього додається функціональний перетворювач, вихід якого через зворотний зв'язок

подається до пам'яті, а також за необхідності входить додатковий блок перетворення.



Рисунок 1.1 Структура ГПВДП

Ключовим елементом генератора є функціональний перетворювач, завдання якого – забезпечити відсутність циклічних повторень у послідовності. Він отримує на вхід  $n$ -компонентний бінарний вектор:

$$X = (X_1, X_2, \dots, X_n), X_i \in \{0, 1\}, i = 1, 2, \dots, n,$$

і на його основі обчислює вихідний вектор  $Y = F(X)$ . До цієї схеми може бути додано вихідний перетворювач, який сприяє забезпеченню необоротності операцій, причому його діяльність часто модифікується за допомогою ключа. Практично цей блок зазвичай репрезентується нелінійним функціональним перетворювачем або селектором бітів, що працює у часовій послідовності.

## 1.2 Критерії ефективності та класифікація ГПВП

Сучасні ГПВП мають відповідати низці фундаментальних критеріїв:

- 1) Ациклічність (розбіжність): алгоритм повинен формувати послідовність двійкових слів довжиною  $n$ , циклічність якої прагне до максимальної величини –  $2^n$ . Це поняття бере свій початок у теорії динамічних систем і передбачає, що повторення елементів відбувається з максимально можливим інтервалом [5].
- 2) Нелінійність: функції перетворення, що використовуються в алгоритмі, мають бути нелінійними. Така властивість ускладнює аналіз та відновлення алгоритмічних зв'язків, що використовуються при генерації [1].

- 3) Відсутність аналітичної залежності: послідовно згенеровані елементи не повинні мати очевидних або прямолінійних математичних взаємозв'язків, що гарантує їх статистичну незалежність.
- 4) Статистична однорідність: ймовірності появи розрядів 0 і 1 мають залишатися рівними (по 1/2 для кожного значення), тобто кожне нове значення формується з однаковою ймовірністю.
- 5) Непередбачуваність: генератор повинен бути настільки складним, що спеціально розроблений алгоритм-розпізнавач не зможе відрізнити його вихід від істинно випадкової послідовності, що забезпечує високий рівень непередбачуваності [6, 7].
- 6) Обчислювальна складність: за визначенням Колмогорова [8], алгоритми мають бути такими, що їх не можна суттєво стиснути математично, що свідчить про складність процедур, які використовуються для генерації послідовності.

ГПВП поділяються на дві великі групи: криптографічні та некриптографічні.

До криптографічних відносяться генератори [1], які побудовані на:

- 1) функціях поточкових шифрів із використанням ключової функції  $E_k$ ;
- 2) функціях блокових шифрів із відповідною криптографічною функцією  $E_k$ ;
- 3) односторонніх функціях, що забезпечують ускладнення зворотного обчислення;
- 4) блоках стохастичного перетворення, які впроваджують додаткове випадкове «зашумлення».

Особлива вимога до криптографічних генераторів полягає у складності відновлення їх внутрішньої логіки, тобто практично неможливості отримати зворотну функцію  $F_k$ . Це означає, що у стійкому криптографічному алгоритмі зазначені властивості гарантують, що неможливо прогнозувати як попередні, так і наступні елементи послідовності. Роль нелінійної функції може виконувати процедура шифрування  $E_k$  у рамках певного алгоритму, що автоматично забезпечує високий рівень криптостійкості ГПВП.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 8    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

До некриптографічних відносять генератори [9], які зазвичай використовуються в застосуваннях, де криптографічна безпека не є критичною, зокрема:

- 1) конгруентні генератори різних принципів (лінійні, степенні тощо);
- 2) генератори, що працюють на основі регістрів зсуву з лінійним зворотним зв'язком;
- 3) системи генерації, засновані на властивостях скінченних полів.

### 1.3 Огляд архітектури та алгоритмів існуючих ГПВП

#### 1.3.1 Лінійний конгруентний ГПВП

Загальна схема лінійних конгруентних генераторів (ЛКГ) була запропонована у 1949 році [10].

Найчастіше на практиці використовуються мультиплікативні конгруентні генератори - генератори, що мають в основі операцію множення. Існує кілька варіантів реалізації таких генераторів. Найчастіше функціональне перетворення, реалізоване в мультиплікативному генераторі, має наступний вигляд:

$$x_n = \left( a^k x_{n-k} + \frac{a^k - 1}{a - 1} b \right) \bmod m,$$

де  $x_n$  —  $n$ -й член послідовності,  $x_{n-k}$  — член послідовності, що відстає від даного на  $k$  елементів;  $a$ ,  $b$ ,  $m$  — постійні коефіцієнти:  $a$  — множник,  $b$  — інкрементор (приріст),  $m$  — модуль. Ключем слугує значення  $x_0$ .

Вибір початкового значення і коефіцієнтів відбувається наступним чином:

- 1) модуль  $m > 0$
- 2) множник  $0 \leq a < m$
- 3) приріст  $0 \leq b < m$
- 4) початкове значення  $0 \leq x_0 < m$

Сформована послідовність  $\{x_n\}$  буде лінійною конгруентною. Максимальний період такої послідовності не перевищує  $m$ . Найчастіше на практиці параметр  $m$  є ступенем 2, так що операція взяття результату за модулем  $m$  зводиться до відкидання старших розрядів.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 9    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Лінійна конгруентна послідовність, визначена числами  $m, a, b$  та  $x_n$ , має період завдовжки  $m$  тоді і тільки тоді, коли:

- 1) числа  $b$  та  $m$  є взаємно простими числами;
- 2)  $(a - 1) : p$  для всіх простих значень  $p$ , для яких  $m : p$ ;
- 3)  $(a - 1) : 4$ , якщо  $m : 4$ .

Дослідження показують, що основними недоліками лінійного конгруентного генератора є невисока ациклічність та недостатньо якісні статистичні показники. Крім цього, якщо ймовірність рівності одиниці старших розрядів результату близька до 0.5, то для молодших розрядів ця умова не виконується. Серйозними недоліками є оборотність та невисока ациклічність. Тому на практиці генератори цього типу використовуються лише для статистичного моделювання (оскільки вони забезпечують достатньо високу швидкодію) і непридатні для цілей криптографії.

### 1.3.2 Степенний конгруентний ГПВП

Значно якіснішим порівняно з ЛКГ є генератор, побудований на основі операцій піднесення до степеня, в якому реалізується наступне функціональне перетворення:  $x_{n+1} = (x_n)^d \bmod n$  [9].

Це є частковим випадком поліноміального конгруентного генератора. Відомо, що якість генератора підвищується зі зростанням значення  $d$ . Існують два важливі часткові випадки степенного генератора, що мають місце, коли  $n = p_1 \cdot p_2$ , де  $p_1$  і  $p_2$  — прості числа. У цьому випадку степенний генератор є добре відомим RSA-генератором, який широко використовується в сучасній криптографії. Основною особливістю RSA-генератора є взаємнооднозначність перетворення, що забезпечує високу ациклічність генераторів цього типу.

Іншим частковим випадком степенного генератора є квадратичний генератор, який реалізує функцію:  $x_{n+1} = (x_n)^2 \bmod n$

Цей тип генератора є найпростішим степенним генератором, при використанні якого забезпечується найбільша продуктивність. Загальним недоліком степенних генераторів є відносно мала продуктивність, пов'язана з трудомісткістю багаторазового піднесення до степеня. Крім того, зазвичай

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 10   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

розрядність проміжних результатів у степенних генераторах значно перевищує розрядність універсального мікропроцесора.

### 1.3.3 Поліноміальний конгруентний ГПВП

Цей генератор розроблявся як спроба поліпшити ймовірнісні характеристики генерованої псевдовипадкової послідовності лінійного конгруентного генератора [9, 11]. Рівняння роботи поліноміального конгруентного ГПВП має вигляд:

$$x_{n+1} = (a_i x_n^i + \dots + a_1 x_n + b) \bmod m$$

Максимальний період такої послідовності дорівнює  $m$ . Правила підбору чисел  $a_i$ ,  $b$  і  $m$ , при яких досягається це значення, залежать від ступеня полінома і визначаються за спеціальними правилами. Існують спеціальні таблиці з уже готовими значеннями цих параметрів. Наприклад, для квадратичного генератора вони наступні:

- 1) числа  $b$  та  $m$  — взаємно прості;
- 2)  $a_1$  та  $a_2$  — кратні для всіх  $p$  непарних простих дільників  $m$ ;
- 3)  $a_2$  – парне і  $a_2 = (a_1 - 1) \bmod 4$ , якщо  $m$  кратне 4;
- 4)  $a_2 = (a_1 - 1) \bmod 2$ , якщо  $m$  кратне 2;
- 5)  $a_2 \neq 3b \bmod 9$ , якщо  $m$  кратне 9.

Недоліком такого генераторів є відносно мала продуктивність.

### 1.3.4 ГПВП RSA

ГПВП RSA належить до криптографічних генераторів, і побудований аналогічним чином на основі односторонньої функції з секретом, що лежить в основі криптосистеми RSA, тобто на основі функції:  $F(x) = x^e \bmod n$ , де  $n = pq$ ;  $p$  і  $q$  – прості числа, причому  $p \neq q$ , а число  $e$  – взаємно просте з  $(p-1)(q-1)$  [1, 12].

Розглянемо алгоритм отримання псевдовипадкової послідовності за допомогою генератора RSA:

- 1) Виберемо як ініціалізуючий вектор випадкове число  $x_0 \in Z_n^*$ .
- 2) Шуканою послідовністю бітів довжиною  $m$  буде послідовність:

$$RSA_{(n,m)}(x_0) = b_0 b_1 b_2 \dots, b_i, \dots, b_{(m-1)}, i = 0, (m-1),$$

де  $b_i$  – молодший біт числа  $x_i$

$$x_{i+1} = x_i^e \bmod n.$$

Захищеність генератора RSA ґрунтується на труднощі розкладання значення  $n$  на множники, тобто на труднощі задачі знаходження простих множників  $p$  та  $q$  за даним  $n$ .

### 1.3.5 ГПВП на регістрах зсуву з лінійними зворотними зв'язками

Серед ГПВП особливе місце займають ті, що реалізуються за допомогою регістрів зсуву з лінійними зворотними зв'язками (LFSR). Їхня основна перевага полягає в тому, що завдяки спеціально розробленим методам синтезу функцій зворотного зв'язку досягається ациклічність роботи — для регістру зсуву з  $n$  розрядами цикл повторення дорівнює  $2^n - 1$  [5]. Додатковою перевагою є відносна простота схемної реалізації апаратними засобами та висока швидкодія.

Структурно такий генератор складається з двох компонентів: функції зворотнього зв'язку, та власне самого регістру зсуву. Останній є послідовністю бітів, при кожному зверненні до нього для отримання нового біта всі існуючі біти пересуваються на одну позицію, а новий крайній біт обчислюється як функція від решти бітів. Зазвичай на виході отримується один біт (часто саме молодший). Період роботи регістру визначається як довжина утвореної послідовності до моменту її циклічного повторення.

Для оцінки ефективності генераторів, побудованих на базі LFSR, застосовується поняття лінійної складності. Вона визначається як довжина найменшого LFSR, здатного відтворити генераторську послідовність. Будь-яка послідовність, сформована кінцевим автоматом над скінченним полем, володіє скінченною лінійною складністю, що має важливе практичне значення, адже існує ефективний алгоритм, який, дослідивши лише  $2n$  бітів потоку, може відновити відповідний LFSR.

Для аналізу LFSR застосовують математичний апарат, який об'єднує теорію лінійних послідовнісних машин та теорію скінченних полів (полів Галуа). Найбільш відомими застосуваннями цієї методології є, зокрема, CRC-коди — ефективний засіб перевірки цілісності даних при їхніх випадкових спотвореннях,

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 12   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

реалізація концепцій самотестування великих інтегральних схем, а також використання у потікових шифрах типів A5, PANAMA, SOBER, SNOW і в блочному шифрі RIJNDAEL (AES), прийнятому у 2001 році як стандарт криптографічного захисту XXI століття.

Основною інформацією, яка визначає роботу двійкового LFSR, є утворюючий многочлен: його ступінь  $n$  задає розрядність регістру зсуву, а ненульові коефіцієнти визначають схему зворотних зв'язків.

Оскільки кількість усіх можливих комбінацій із  $n$  двійкових розрядів дорівнює  $2^n$ , період згенерованої послідовності не може перевищувати це значення. При цьому, якщо LFSR опиниться в стані, коли всі біти рівні нулю, він не зможе далі змінювати свій стан, тому при ненульовому початковому векторі виникнення нульового стану повинно бути виключене. В результаті максимальна кількість можливих станів обмежується значенням  $2^n - 1$ .

Послідовність максимальної довжини формується тоді, коли функція зворотного зв'язку реалізована відповідно до простого поліному на полях Галуа.

Усі елементи поля Галуа, за винятком нуля, можуть бути згенеровані за допомогою LFSR. Існує два способи представити простий поліном як LFSR. Нижче показана конфігурація, що називається простим генератором на регістрі зсуву (SSRG) [13].

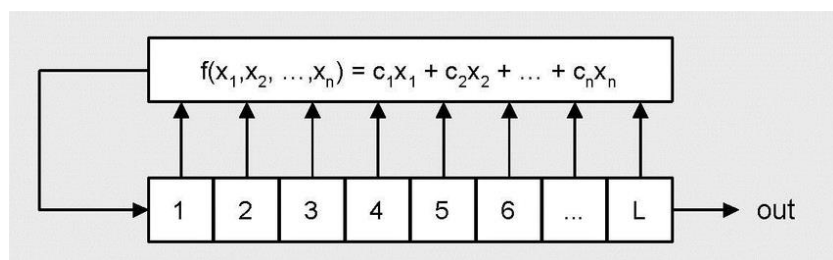


Рисунок 1.3 Простий генератор на зсувному регістрі

Однак генератори, побудовані виключно на регістрах зсуву з лінійною функцією зворотного зв'язку, не відповідають вимогам до складності та необоротності, адже їх робота описується лінійними булевими функціями. Це означає, що, маючи  $2n$  бітів згенерованої послідовності, можна шляхом



розв'язання системи з  $2n$  лінійних булевих рівнянь однозначно відновити початковий  $n$ -бітовий код регістру та конфігурацію функції зворотного зв'язку.

Виходячи з цього, реальні ГПВП, призначені для застосування в системах захисту інформації, містять суттєво складнішу архітектуру, у якій обов'язковим елементом є нелінійні булеві перетворювачі [14]. При такій структурі математична безпека системи еквівалентна задачі розв'язання систем нелінійних булевих рівнянь, яку неможливо вирішити аналітичними методами — єдиним варіантом є перебір усіх можливих комбінацій. Найпоширенішими практичними реалізаціями генераторів двійкових послідовностей, що поєднують роботу LFSR та нелінійних булевих перетворень, є мультиплексні, фільтруючі та табличні генератори [1].

### 1.3.6 ГПВДП довільної довжини

Недоліком ГПВП на основі регістрів зсуву з лінійною функцією зворотного зв'язку є те, що нульовий стан для них заборонений. Через це ймовірність випадання 0 і 1 неоднакова.

Щоб збільшити період сформованої послідовності та зробити його максимально можливим, рівним  $2^N$ , і підвищити її якість, рівняння роботи генератора з лінійним зворотним зв'язком перетворено до наступного вигляду:

$$q_0(t+1) = \overline{q_0(t) \cdot q_1(t) \cdot \dots \cdot q_{(N-2)}(t)} \oplus \sum_{i=1}^N a_i * q_{(i-1)}(t) \pmod{2}$$

$$q_j(t+1) = q_{(j-1)}(t), j = \overline{1, N-1}$$

Генератор послідовності довільної довжини  $2^N - 1 < S < 2^N - 1$  будується таким чином. Обирається примітивний многочлен  $\Phi(x)$  ступеня  $N$ . Фіксується довільний ненульовий стан  $Q_0$  генератора. Моделюємо  $t = 2^N - S$  тактів роботи ГПВП на основі регістрів зсуву з лінійною функцією зворотного зв'язку і визначаємо стан  $Q_t$ . Виконуємо порозрядну операцію XOR над кодами  $Q_1$  та  $Q_t$ .

Одиничні біти результату визначають номери тих тригерів ГПВП, сигнали на D-входах яких необхідно інвертувати, коли генератор перебуває у стані  $Q_0$ . Керовані інвертори реалізуються на додаткових елементах XOR, число яких та місце в схемі генератора визначаються результатом операції  $Q_t \oplus Q_0$ .

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 14   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

### 1.3.7 Мультиплексні ГПВП

Мультиплексна схема формування псевдовипадкової послідовності складається з двох базових регістрів зсуву, кожен з яких має різну схему лінійного зворотного зв'язку, регістр маски, який підключається разом з виходами базових регістрів зсуву до входів логічних елементів І, виходи логічних елементів для першого і другого базових регістрів під'єднані до входів багатовходових суматорів за модулем 2, виходи першого з них –  $\alpha$  і другого  $\beta$  – зв'язані зі входами мультиплексора та суматора за модулем 2, вихід якого керує мультиплексором вибору джерела вихідного сигналу. Для підвищення стійкості до розкриття у схемі, як і в прототипі, використовується тимчасовий селектор, реалізований лічильником, який відбирає на вихід лише кожен 64-й біт генерованої двійкової послідовності [1].

Недоліком розглянутої схеми є необхідність збільшення вдвічі порівняно з прототипом передаваної синхронізуючої інформації (для першого та другого регістрів), а також практично вдвічі складніша схема. Водночас схема з двома регістрами зсуву також залишається недостатньо нелінійною.

### 1.3.8 ГПВП Геффа

ГПВП Геффа, або Джиффі [15], який є різновидом мультиплексного генератора, будується на основі трьох LFSR, як показано на рис. 1.4 [16].

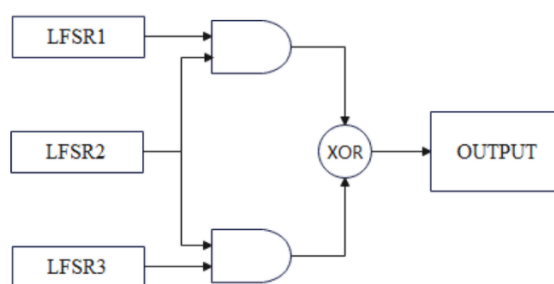


Рисунок 1.4 Генератор Геффа

У поданій схемі два елементи виступають як входи мультиплексора, тоді як третій визначає його вихідний сигнал. Якщо 3 LFSR мають виходи  $x_1, x_2, x_3$ , тоді генератор Геффа можна представити так:

$$F(x_1, x_2, x_3) = \overline{x_1 x_3} + x_2 x_3 = x_3 \oplus x_1 x_2 \oplus x_2 x_3.$$

При цьому можливі альтернативні варіанти реалізації. Наприклад, застосування функції  $F(x_1, x_2, x_3) = x_1x_2 \oplus x_2x_3 \oplus x_1x_3$  дозволяє отримати послідовності з вищою лінійною складністю.

Для генерації  $n$ -розрядного вихідного сигналу можна покращити процес змішування двох  $n$ -бітних псевдовипадкових послідовностей шляхом використання функції ускладнення, що ґрунтується на  $n$  однобітних мультиплексорах типу 2:1. На відміну від класичного вибору між двома LFSR, ця схема передбачає вибір серед  $k$  LFSR, де  $k$  є степенем двійки. Загалом використовується  $(k+1)$  регістрів, причому тактова частота першого LFSR повинна бути вищою, ніж у решти  $k$  регістрів.

### 1.3.9 Старт-стопний ГПВП

Старт-стопний генератор використовує вихід першого LFSR для управління тактовою частотою другого, так що LFSR 2 може змінювати свій стан у момент часу  $t$  лише якщо в момент часу  $t-1$  вихід LFSR 1 був рівний одиниці.

У чергується старт-стопному генераторі використовуються три LFSR різної довжини. LFSR 2 тактується, коли вихід LFSR 1 дорівнює 1, LFSR 3 тактується, коли вихід LFSR 1 дорівнює 0. Виходом генератора є  $(\text{LFSR 2} \oplus \text{LFSR 3})$ . Цей генератор має великий період та велику лінійну складність.

Застосування старт-стопних схем генераторів [6] у системах захисту інформації є неефективним, оскільки при їх використанні код маски має бути пов'язаний зі схемою генерації сигналів зсуву. Разом з тим, виходячи з вимоги забезпечення синхронізації на передавачі та приймачі, регістри на них повинні зсуватися строго синхронно, що, своєю чергою, вимагає передачі синхронізуючої інформації, яка в даній схемі може бути використана для розкриття коду маски.

Каскадний генератор Голлмана (рис. 1.5) являє собою посилену версію старт-стопного генератора [17]. Він складається з послідовності LFSR, тактування кожного з яких керується попереднім LFSR. Такі генератори можуть бути використані для генерації послідовностей з дуже великими лінійними складностями та з великими періодами.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 16   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

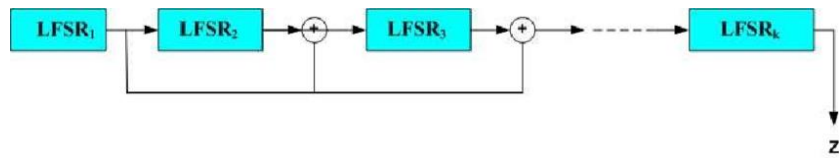


Рисунок 1.5 Каскадний генератор Голлмана

## 1.4 Аналіз використання ГПВП у комп'ютерних системах

Генератори випадкових та псевдовипадкових послідовностей використовуються для вирішення наступних завдань [1, 9]:

- 1) кодування інформації при передачі даних у різних каналах зв'язку;
- 2) хешування даних;
- 3) генерування секретних ключів для систем захисту інформації;
- 4) імітація випадкових джерел впливу в системах моделювання;
- 5) тестування різних пристроїв та програмних продуктів;
- 6) формування CRC-кодів.

Як критерії якості ГПВП найчастіше використовуються такі показники [18]:

- 1) статистичні критерії, що дозволяють оцінити ймовірнісні характеристики згенерованої послідовності;
- 2) критерії нециклічності або розбіжності роботи генератора, що оцінюються мінімальною довжиною циклу, який виникає при роботі генератора;
- 3) непередбачуваність роботи генератора, яка оцінюється складністю спеціального алгоритму-розпізнавача, що дозволяє відрізнити генеровану послідовність від випадкової;
- 4) продуктивність генератора.

При розгляді цих критеріїв важливим є принцип, сформульований у [6] - критерій визначення псевдовипадковості завжди визначається класом завдань, у якому буде використана ця псевдовипадковість, тобто використання повинно визначати значущість кожного з наведених вище критеріїв. Для задач статистичного моделювання головне значення мають статистичні характеристики, тоді як у системах захисту інформації основними критеріями якості ГПВДП є властивості, що забезпечують криптологічну стійкість.

## ВИСНОВОК ДО РОЗДІЛУ 1

У задачах захисту інформації, а також при статистичному контролі та діагностиці, моделюванні та проектуванні широко застосовуються генератори псевдовипадкових двійкових послідовностей. Ефективність генераторів псевдовипадкових об'єктів в інформаційних технологіях визначається конкретними особливостями їх використання. Зокрема, для задач моделювання та технічної діагностики домінуючу роль відіграють статистичні характеристики (вид статистичного розподілу, стабільність його характеристик). На відміну від вищезгаданих областей використання псевдовипадкових об'єктів, у системах захисту інформації основними критеріями якості ГПВДП є необоротність та складність побудови відтворюючої моделі за заданим фрагментом послідовності [19], тобто критерії, що визначають рівень захищеності даних. Важливою характеристикою якості ГПВП є ефективність реалізації апаратними засобами. Справа в тому, що більшість практичних застосувань потокових алгоритмів захисту даних, основним елементом яких є ГПВП, передбачає їх апаратну реалізацію. Визначальним фактором ефективності апаратної реалізації є швидкодія та апаратна складність генератора.

В межах розділу було проаналізовано основні типи генераторів, що широко використовуються на сьогоднішній день, та критерії оцінки їхньої якості.

Найчастіше на практиці використовуються конгруентні генератори, але вони мають недостатньо якісні статистичні показники та не забезпечують необоротності, тому непридатні для систем захисту інформації. До того ж конгруентні генератори вимагають виконання порівняно складних операцій множення, що знижує їхню швидкодію.

У системах захисту інформації застосовуються синхронні криптостійкі ГПВП на основі потокових шифрів. Хоча багато криптостійких потокових шифрів пропонують значно кращі показники «випадковості», такі генератори набагато повільніші.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 18   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

З точки зору продуктивності більш прийнятними є генератори на основі регістрів зсуву зі зворотним зв'язком. Широко використовуються генератори на регістрах зсуву з лінійною функцією зворотного зв'язку, але, по-перше, знаходження лінійних функцій з максимальним періодом є непростим завданням; по-друге, загальна кількість придатних функцій недостатньо велика. Крім того, лінійні функції не відповідають вимогам до рівня захищеності інформації.

Тому найбільш доцільним є побудова ГПВДП на основі регістру зсуву з нелінійною функцією зворотного зв'язку. При цьому необхідно забезпечити вирішення ряду проблем:

- 1) у вихідній послідовності не повинно бути зміщень, коли, наприклад, кількість одиниць більша, ніж нулів;
- 2) період повторення послідовності повинен бути якомога більшим;
- 3) період повинен бути не меншим за заданий при виборі будь-яких початкових значень.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 19   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

## РОЗДІЛ 2

# ТЕОРЕТИЧНЕ ДОСЛІДЖЕННЯ ВЛАСТИВОСТЕЙ НЕЛІНІЙНИХ ФУНКЦІЙ ЗВОРОТНЬОГО ЗВ'ЯЗКУ ДЛЯ ЗСУВНОГО РЕГІСТРУ, ЩО ЗАБЕЗПЕЧУЮТЬ ПОВНИЙ ПЕРІОД

### 2.1 Загальні властивості нелінійних функцій зворотного зв'язку, що забезпечують максимальний період

ГПВП широко використовуються в інформаційних технологіях, зокрема для захисту даних, що передаються через відкриті ефірні канали. При цьому необхідно забезпечити достатньо високий рівень захищеності. Ця вимога досягається за рахунок підвищення нелінійності функціонального зв'язку між бітами двійкової послідовності, яка використовується в потокових алгоритмах шифрування даних. Важливою задачею з точки зору підвищення рівня захисту є створення ГПВП на основі зсувного реєстра з нелінійною функцією зворотного зв'язку.

При виборі функції зворотного зв'язку випадковим чином не забезпечується прийнятний для практичних застосувань період повторення двійкових послідовностей [20]. Кількість функцій від  $n$  аргументів визначається формулою:

$$K = 2^{2^{n-1} - n}$$

Загальна кількість всіх функцій становить  $2^{2^n}$ . Отже, в середньому одна функція з повним періодом припадає на  $2^{2^{n-1} + n}$  функцій. А згідно з сучасними стандартами для систем мобільного зв'язку  $n \geq 100$ . На відміну від лінійних функцій, проблема синтезу нелінійних функцій, що забезпечують повний період, на даний час не розв'язана. В цей же час значні зусилля дослідників у цій галузі спрямовані на розробку ефективних підходів для побудови таких генераторів, в тому числі для телекомунікаційних систем [21-23].

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 20   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Загальні критерії, такі як ациклічність, непередбачуваність та статистична однорідність, є невід'ємними для забезпечення криптографічної міцності будь-якого генератора. Ациклічність гарантує, що послідовність не повторюватиметься завчасно, забезпечуючи довгий період. Непередбачуваність, яка часто оцінюється складністю алгоритму-розпізнавача, є ключовою для запобігання атакам, де супротивник намагається передбачити майбутні або минулі значення послідовності. Статистична однорідність (рівні ймовірності 0 та 1) запобігає появі будь-яких статистичних упереджень, які можуть бути використані для криптоаналізу. Нелінійність функцій перетворення безпосередньо впливає на виконання цих критеріїв. Вона ускладнює відновлення внутрішньої логіки генератора за фрагментом вихідної послідовності, роблячи його стійкішим до лінійних та алгебраїчних атак, які часто ефективні проти LFSR.

Для криптографічних застосувань, де необхідна висока непередбачуваність та відсутність статистичних аномалій, застосування нелінійних перетворень є імперативом. Застосування алгебраїчної нормальної форми (АНФ) дозволяє більш глибоко аналізувати структуру булевих функцій, а їхня нелінійність в АНФ слугує мірою складності розкриття функції, визначаючи, наскільки вона віддалена від лінійних функцій [24, 25].

Розглянемо властивості, якими володіють нелінійні функції зворотного зв'язку, що забезпечують повний період.

Стан зсувного регістра характеризується кодом  $W$ , якому відповідає двійковий вектор  $X_W$  значень розрядів регістра:

$$X_W = \{X_0^W, X_1^W, \dots, X_{n-2}^W, X_{n-1}^W\}$$

Структура зсувного регістра з нелінійним зворотним зв'язком показана на рис. 2.1.

Теорема: нелінійна булева функція  $f(x_0, x_1, \dots, x_{n-1})$  зворотного зв'язку NLFSR повинна приймати протилежні значення на наборах змінних (кодах зсувного регістра), що відрізняються значеннями старшого розряду  $X_0$ :

$$f(x_0, x_1, \dots, x_{n-1}) = 1 \oplus f(1 \oplus x_0, x_1, \dots, x_{n-1}) \quad (2.1)$$

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 21   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |



тобто в алгебраїчній нормальній формі (алгебрі Жегалкіна) функція може бути представлена як сума за модулем 2 змінної, що відповідає старшому розряду зсувного регістра  $x_0$  та нелінійної функції  $\varphi(x_1, \dots, x_{n-1})$ , що не залежить від  $x_0$ :  $f(x_0, x_1, \dots, x_{n-1}) = x_0 \oplus \varphi(x_1, \dots, x_{n-1})$ .

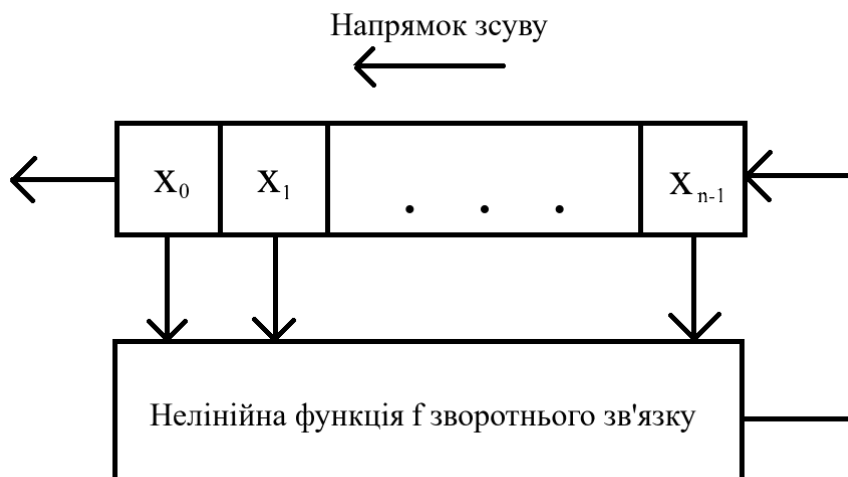


Рисунок 2.1 Зсувний регістр з нелінійним зворотним зв'язком

Доведемо раніше наведену теорему: необхідною умовою для побудови повноциклічного генератора псевдовипадкових двійкових послідовностей на основі зсувного регістра з нелінійною функцією зворотного зв'язку є однозначність формування кодів на регістрі: тобто кожному з  $2^n$  можливих кодів регістра повинен передувати лише один попередній код, або булева функція зворотного зв'язку повинна бути однозначною.

Очевидно, що якщо за  $2^n$  циклів роботи генератора не буде повторно сформовано жодного коду, це означатиме, що за  $2^n$  циклів буде сформовано всі можливі коди від 0 до  $2^n - 1$ . Код  $j$  на зсувному регістрі може бути сформований двома способами: якщо номер попереднього коду дорівнює  $\frac{j}{2}$ , або  $(2^{n-1} + \frac{j}{2})$ :

$$j = 2\left(\frac{j}{2}\right) + f\left(\frac{j}{2}\right)$$

$$j = 2\left(\frac{j}{2} + 2^{n-1}\right) - 2^n + f\left(\frac{j}{2} + 2^{n-1}\right)$$

Зрозуміло, що для непарного  $j$  значення функції  $f\left(\frac{j}{2}\right)$  та  $f\left(\frac{j}{2} + 2^{n-1}\right)$  повинні дорівнювати нулю, а для парного  $j$  – одиниці. Для того щоб код на

зсувному реєстрі не зміг бути згенерований повторно протягом циклу, необхідно, щоб значення функції зворотного зв'язку на кодах  $\frac{j}{2}$  та  $\frac{j}{2} + 2^{n-1}$  завжди були протилежними, тобто

$$f\left(\frac{j}{2}\right) \neq f\left(\frac{j}{2} + 2^{n-1}\right) \oplus 1,$$

або враховуючи, що  $\frac{j}{2} < 2^{n-1}$ :

$$f(x_0, x_1, \dots, x_{n-1}) = 1 \oplus f(1 \oplus x_0, x_1, \dots, x_{n-1})$$

Це означає, що кожен код  $j$  на зсувному реєстрі може бути сформований лише одним способом, і необхідною умовою для повторного формування коду  $j$  є повторне формування коду, що йому передує.

Глибокий аналіз умови (2.1) виявляє її фундаментальне значення для унеможливлення появи локальних циклів, що мали б довжину меншу за  $2^n$ . Це гарантує, що кожен стан реєстра має рівно одного унікального попередника, створюючи бієкцію між простором станів і їх наступниками. Таке жорстке правило унеможливорює "розгалуження" або "злиття" шляхів обходу станів, що призводило б до декількох входів до одного стану або тупикових гілок, які не охоплюють весь простір станів.

Аналогічно, умова (2.2) є запобіжним заходом проти "тривіальних" циклів. Якщо  $f(0, \dots, 0) = 0$ , реєстр застряг би в нескінченній послідовності нулів, так само як  $f(1, \dots, 1) = 1$  призвів би до нескінченної послідовності одиниць. Ці умови, в сукупності, забезпечують, що шлях, розпочатий з будь-якого стану (крім заборонених, як  $00\dots 0$ ), або шлях, розпочатий з  $00\dots 0$  і коректно ініціалізований, зрештою пройде через всі  $2^n$  можливих станів, утворюючи єдиний і максимальний за довжиною цикл. Цей принцип є фундаментальним для проектування надійних NLFSR.

З цього буде впливати наступне: якщо представити функцію, що задовольняє наведену вище вимогу, у вигляді таблиці істинності, то значення на наборах з її верхньої половини завжди будуть рівні інверсії відповідних наборів з нижньої половини.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 23   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

У табл. 2.1 наведено всі функції, що забезпечують повний період  $2^n$  для довжини регістра  $n = 4$ . Як буде показано далі, всього існує 16 таких функцій.

Таблиця 2.1 – Функції F1-F16, що забезпечують повний період для  $n = 4$

| Число     | F1 | F2 | F3 | F4 | F5 | F6 | F7 | F8 | F9 | F10 | F11 | F12 | F13 | F14 | F15 | F16 |
|-----------|----|----|----|----|----|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|
| 0000 (0)  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1   | 1   | 1   | 1   | 1   | 1   | 1   |
| 0001 (1)  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 1   | 1   | 1   | 1   | 1   | 1   | 1   |
| 0010 (2)  | 0  | 0  | 1  | 1  | 1  | 1  | 1  | 1  | 0  | 0   | 1   | 1   | 1   | 1   | 1   | 1   |
| 0011 (3)  | 0  | 1  | 0  | 0  | 0  | 1  | 1  | 1  | 0  | 1   | 0   | 0   | 0   | 1   | 1   | 1   |
| 0100 (4)  | 1  | 1  | 0  | 1  | 1  | 0  | 1  | 1  | 0  | 0   | 0   | 0   | 1   | 0   | 0   | 1   |
| 0101 (5)  | 1  | 1  | 1  | 0  | 1  | 1  | 0  | 1  | 1  | 1   | 0   | 1   | 1   | 0   | 1   | 1   |
| 0110 (6)  | 1  | 0  | 1  | 1  | 0  | 0  | 0  | 1  | 1  | 0   | 1   | 0   | 1   | 0   | 1   | 0   |
| 0111 (7)  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1   | 1   | 1   | 1   | 1   | 1   | 1   |
| 1000 (8)  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 1001 (9)  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 0  | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 1010 (10) | 1  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 1   | 0   | 0   | 0   | 0   | 0   | 0   |
| 1011 (11) | 1  | 0  | 1  | 1  | 1  | 0  | 0  | 0  | 1  | 0   | 1   | 1   | 1   | 0   | 0   | 0   |
| 1100 (12) | 0  | 0  | 1  | 0  | 0  | 1  | 0  | 0  | 1  | 1   | 1   | 1   | 0   | 1   | 1   | 0   |
| 1101 (13) | 0  | 0  | 0  | 1  | 0  | 0  | 1  | 0  | 0  | 0   | 1   | 0   | 0   | 1   | 0   | 0   |
| 1110 (14) | 0  | 0  | 0  | 0  | 1  | 1  | 1  | 0  | 0  | 1   | 0   | 1   | 0   | 1   | 0   | 1   |
| 1111 (15) | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   | 0   | 0   | 0   | 0   | 0   | 0   |

З таблиці видно, що для всіх F1 – F16 виконується умова (2.1): таблиця істинності “симетрична”, значення функцій на наборах з номерами 8 – 15 інверсні по відношенню до значень на відповідних наборах з номерами 0 – 7.

Легко помітити, що на наборі з номером нуль функція повинна дорівнювати одиниці. В іншому випадку, якщо зсувний регістр потрапить у нульовий стан, він буде генерувати нескінченну послідовність нулів. Те ж саме стосується набору з номером  $2^n - 1$  (при  $n = 4$  – набір 15).

Усі розряди регістра будуть рівні одиниці, і щоб забезпечити перехід в інший стан, функція повинна повертати нульове значення. Отже,

$$f(0,0, \dots, 0) = f(0) = 1,$$

$$f(1,1, \dots, 1) = f(2^n - 1) = 0 \quad (2.2)$$

Внаслідок цих двох властивостей усі функції, що забезпечують повний період, також повинні приймати одиничне значення на наборі  $2^{n-1} - 1$  (011...1) і нульове значення на наборі  $2^n - 1$  (100...0). Для  $n = 4$  кожна з функцій приймає одиничне значення на наборі 7 і нульове значення на наборі 8.

З цього випливає, що при аналізі таблиці істинності можна розглядати не всі  $2^n$  наборів, а тільки  $2^{n-1} - 2$  наборів, тобто набори з номерами від 1 до  $2^{n-1} - 2$ . Для  $n = 4$  це номери від 1 до 7 включно з обох боків.

Розглянемо більш детально функції, наведені в таблиці 2.1. Для всіх функцій виконується наступне правило: на наборах з номерами від 1 до  $2^{n-1} - 2$  кількість нулів і, відповідно, одиниць, має бути непарною. Так, функції з номерами 8, 12, 14 і 15 містять у цій області один нуль, а решта – 3 нулі. Аналіз функцій для більшої кількості змінних підтверджує, що всі вони містять непарну кількість нулів (одиниць) у даній області.

Характерною є поведінка функцій від 4 аргументів на наборах 5 і 10. Якщо на наборі 5 значення функції дорівнює нулю, то і на наборі 10 також повинен бути нуль. Це пояснюється просто: якщо вектор розрядів зсувного регістра матиме вигляд {0101}, що відповідає набору 5, то при нульовому значенні функції після зсуву утворюється вектор {1010} (набір 10). Якщо тепер функція дорівнюватиме одиниці, регістр знову опиниться в стані {0101}. Щоб запобігти нескінченному чергуванню цих двох станів, функція повинна мати нульове значення на наборі 10. Очевидно, ця ситуація має місце і при більшій кількості аргументів.

Функції з повним періодом володіють наступною особливістю: якщо значення функції на наборах виду (0101...01) і (1010...10) різні, то якщо змінити значення функції на цих двох наборах (а також на симетричних їм) на протилежні, то отримана функція теж матиме повний період

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 25   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

## 2.2 Аналіз кількості функцій зворотного зв'язку, які забезпечують максимальний період

Визначимо кількість нелінійних функцій, що забезпечують повний період. Для цього проаналізуємо загальну кількість функцій  $K_n$ , отриману при різних кількості аргументів  $n$ . Зауважимо при цьому, що для будь-якого  $n$  кількість функцій є степенем двійки:  $K_n = 2^{k_n}$ .

Таблиця 2.2 – Значення  $K_n$  і  $k_n$  для деяких  $n$

| $n$   | 2 | 3 | 4  | 5    | 6        |
|-------|---|---|----|------|----------|
| $K_n$ | 1 | 2 | 16 | 2048 | 67108864 |
| $k_n$ | 0 | 1 | 4  | 11   | 26       |

Визначимо різницю  $d_n = k_n - k_{n-1}$  між усіма сусідніми значеннями  $k_n$ ,  $n = 3 \dots 6$ , взяті попарно:

$$d_3 = k_3 - k_2 = 1 = 2^1 - 1$$

$$d_4 = k_4 - k_3 = 3 = 2^2 - 1$$

$$d_5 = k_5 - k_4 = 7 = 2^3 - 1$$

$$d_6 = k_6 - k_5 = 15 = 2^4 - 1$$

Використовуючи отримані значення  $d_n$ , визначимо значення  $k_n$ ,  $n = 2, \dots, 6$  наступним чином:

$$k_2 = 2^0;$$

$$k_3 = k_2 + d_3 = 2^0 + 2^1 - 1;$$

$$k_4 = k_3 + d_4 = 2^0 + 2^1 + 2^2 - 1$$

$$k_5 = k_4 + d_5 = 2^0 + 2^1 + 2^2 + 2^3 - 1$$

$$k_6 = k_5 + d_6 = 2^0 + 2^1 + 2^2 + 2^3 + 2^4 - 1$$

Припустивши, що така закономірність зберігається для всіх  $n$ , приходимо до наступної формули визначення кількості нелінійних функцій зворотного зв'язку, що забезпечують повний період:

$$K_n = 2^{k_n}, \text{ де } k_n = 2^0 + 2^1 + \dots + 2^{n-1} - 1 = \sum_{i=0}^{n-1} 2^i - 1$$

Кількість нелінійних функцій, що забезпечують максимальний період, і відповідно кількість генераторів, які можуть бути побудовані з їх використанням, значно більше кількості генераторів з лінійними функціями зворотного зв'язку.

Наприклад, при 6-розрядному зсувному регістрі можна отримати 67108864 різних нелінійних функцій і, відповідно, побудувати 67108864 генераторів, тоді як кількість генераторів з лінійними функціями зворотного зв'язку дорівнює 6.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 27   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

## ВИСНОВОК ДО РОЗДІЛУ 2

У цьому розділі було проведено теоретичне дослідження властивостей нелінійних функцій зворотного зв'язку, що є ключовим елементом генераторів ГПВДП для забезпечення повного періоду їх роботи. Зокрема, сформульовано та доведено необхідні умови до функції зворотного зв'язку, яка полягає у прийнятті протилежних значень на наборах змінних, що відрізняються лише значенням старшого розряду. Це гарантує однозначне формування наступного стану регістра та виключає утворення локальних циклів меншої довжини.

Також було показано обов'язкові значення функції для особливих наборів, таких як повністю нульовий та повністю одиничний вектори стану, що є критично важливими для запобігання дегенерації послідовності в нескінченні ланцюги нулів або одиниць. Визначені умови дозволяють значно звузити простір пошуку функцій зворотного зв'язку, придатних для створення повноциклічних генераторів.

Крім того, проведений аналіз кількості таких функцій засвідчив, що число нелінійних функцій зворотного зв'язку, здатних забезпечити максимальний період ( $K_n = 2^{k_n}$ ), є експоненційно більшим порівняно з кількістю лінійних повноперіодних функцій, що є важливим для використання у складних архітектурах [26]. Цей результат відкриває значно ширші можливості для конструювання криптографічно стійких ГПВДП, пропонуючи більшу варіативність для проєктування.

Таким чином, теоретичні основи, закладені у цьому розділі, є фундаментальними для розробки конкретних методів синтезу нелінійних функцій зворотного зв'язку в подальших розділах.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 28   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

# РОЗДІЛ 3

## РОЗРОБКА МЕТОДІВ СИНТЕЗУ ГПВДП НА ЗСУВНИХ РЕГІСТРАХ З НЕЛІНІЙНИМ ЗВОРОТНІМ ЗВ'ЯЗКОМ

### 3.1 Базова процедура побудови NLFSR на основі об'єднання мінімальних елементів замкнутих кодових груп

#### 3.1.1 Алгоритм синтезу NLFSR

Стан зсувного реєстра характеризується кодом  $W$ , якому відповідає двійковий вектор  $X_W$  значень розрядів реєстра:

$$X_W = \{x_0^W, x_1^W, \dots, x_{n-2}^W, x_{n-1}^W\}$$

При зсуві реєстра нове значення коду  $V$  визначається наступним чином:

$$X_V = \{x_1^W, x_2^W, \dots, x_{n-2}^W, f(x_W)\}, \quad V = (2w) \bmod 2^n$$

Нехай  $f(x_1, x_2, \dots, x_n)$  – булева функція зворотного зв'язку  $n$ -розрядного зсувного реєстра, для якої виконується умова:

$$f(x_0, x_1, \dots, x_{n-1}) = 1 \oplus f(x_0 \oplus 1, x_1, \dots, x_{n-1}) \quad (3.1)$$

Якщо функція  $f(x_1, x_2, \dots, x_n)$  зворотного зв'язку задовольняє умові (3.1), то кожному коду  $w$  на реєстрі передуює лише один код  $w$ .

Кільцем будемо називати множину кодів, які послідовно формуються на зсувному реєстрі з функцією зворотного зв'язку, що задовольняє умові (3.1). Кожен із  $2^n$  можливих кодів на  $n$ -розрядному реєстрі входить в одне з кілець.

Нехай є два кільця  $A$  і  $B$ . Якщо код  $w$  належить кільцю  $A$ , а симетричний йому код  $v = (w + 2^{n-1}) \bmod 2^n$  належить кільцю  $B$ , тоді інвертування значення функції в цих кодах призведе до об'єднання кілець  $A$  і  $B$ .

Доведення: Коду  $w$  відповідає двійковий вектор стану реєстра

$$X_W = \{x_0^W, x_1^W, \dots, x_{n-2}^W, x_{n-1}^W\}$$

Код  $e$  слідує за кодом  $w$  у кільці  $A$ :

$$X_e = \{x_1^W, x_2^W, \dots, x_{n-2}^W, f(X_W)\}$$

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 29   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |



При інвертуванні значення функції зворотного зв'язку на коді  $w$ , наступним після  $w$  буде код  $u$ :

$$X_u = \{x_1^W, x_2^W, \dots, x_{n-2}^W, f(X_W) \oplus 1\}$$

Код  $V$  є симетричним до  $W$ :

$$X_V = \{x_0^W \oplus 1, x_1^W, \dots, x_{n-2}^W, x_{n-1}^W\}$$

Згідно з (3.1):

$$f(X_V) = 1 \oplus f(X_W),$$

тому код  $u$  слідує за кодом  $v$  у кільці  $B$ .

При інвертуванні функції на коді  $V$ , наступним буде код  $e$ :

$$X_e = \{x_1^W, x_2^W, \dots, x_{n-2}^W, f(X_W) \oplus 1\}$$

Таким чином, при інвертуванні функції на симетричних кодах, що належать різним кільцям  $A$  і  $B$ , після коду  $w$  виконується перехід на код  $u \in B$ . Надалі виконується послідовний прохід усіх кодів кільця  $B$ , останнім з яких є код  $v$ . З цього коду здійснюється перехід на код  $e \in A$ . Далі послідовно проходяться всі коди кільця  $A$ . Таким чином, кільця  $A$  і  $B$  об'єднуються в одне кільце.

Запропонований метод синтезу нелінійної функції зворотного зв'язку, яка забезпечує повний період, ґрунтується на базовій процедурі об'єднання кілець, отриманих при циклічному зсуві.

Функція циклічного зсуву зсувного регістра дорівнює старшому розряду поточного коду  $K$ :

$$f(X_n) = \left[ \frac{k}{2^{n-1}} \right] = x_0^k \quad (3.2)$$

Зрозуміло, що функція (3.2) задовольняє умову (3.1). Функція (3.2) утворює  $N_R$  кілець, кожне з яких містить коди з рівною кількістю одиниць.

Позначимо через  $R(k)$  кільце, яке містить код  $K$ , а через  $L(A)$  — число одиниць в кодах циклічного кільця  $A$ . Наприклад, при  $n = 4$  і  $k = 6$ :  $R(k) = \{6(0110), 12(1100), 9(1001), 3(0011)\}$ ,  $L(R(k)) = 2$ .

Кожне з кілець має єдиний мінімальний код. Очевидно, що для будь-якого циклічного кільця  $A \neq R(0)$ , мінімальний код  $q = \min(A)$  є непарним числом

( $x_{n-1}^q = 1$ ). Це означає, що ненульовий мінімальний код  $q$  циклічного кільця завжди може бути представлений у вигляді:  $q = 2d + 1$ .

Метод об'єднання циклічних кілець є основоположним у синтезі NFSR з повним циклом завдяки його здатності детерміновано керувати послідовністю переходів станів. Ідея представленого методу полягає в тому, що весь простір  $2^n$  можливих станів зсувного регістра може бути розбитий на так звані циклічні кільця — замкнуті підмножини станів, що утворюються, коли регістр функціонує за найпростішою функцією зворотного зв'язку  $f(X_K) = X_0^K$  (яка просто копіює старший біт стану). Різні кільця мають різну кількість одиниць у своїх станах, і головне, що ці кільця ізольовані.

Суть базової процедури полягає в модифікації цієї "ізоляції", шляхом змінення одного біту функції зворотного зв'язку для "особливих" станів (мінімальних елементів кілець), тим самим "перекидаючи" перехід з одного кільця до іншого. Забезпечуючи, що кожен перехід буде унікальним, а змінені значення функції дозволяють об'єднати всі окремі кільця в єдиний, величезний цикл, який проходить через всі  $2^n$  станів. Така конструкція гарантує, що регістр не "застрягне" у малому циклі або підмножині станів. Це забезпечує передбачуваний шлях для побудови функції, що гарантує максимальний період.

Базова процедура об'єднання циклічних кілець полягає у виконанні наступної послідовності дій:

1. Довільно вибирається початкове значення поточного коду  $j$ ,  $0 < j < 2^n$ . Лічильник  $h$  кодів, для яких визначено значення функції зворотного зв'язку, встановлюється в одиницю:  $h := 1$ .

2. Обчислюється  $u = (2 \cdot j) \bmod 2^n + 1$ . Якщо обчислений код  $u$  є мінімальним у своєму кільці, тобто  $u = \min(R(u))$ , то значення функції зворотного зв'язку на коді  $j$  визначається як інверсія циклічного зсуву:

$$f(X_j) = \left\lfloor \frac{j}{2^{n-1}} \right\rfloor \oplus 1 = X_0^j \oplus 1,$$

а в протилежному випадку:

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 31   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

$$f(X_j) = \left\lceil \frac{j}{2^{n-1}} \right\rceil = X_0^j, \text{ де } [x] - \text{найменше ціле, що більше за } x.$$

3. Обчислюється нове значення поточного коду:

$j := (2 \cdot j) \bmod 2^n + f(X_j)$ , після чого виконується інкремент лічильника:  
 $h := h + 1$ . Якщо  $h \leq 2^n$ , то здійснюється повернення на пункт 2 процедури,  
 інакше — завершення процедури.

Функція циклічного перенесення утворює  $N_R$  кілець. Наведена вище процедура забезпечує з'єднання всіх цих кілець в одне.

Доведення: З'єднання кілець здійснюється попарно. Розглянемо довільне кільце  $B$ , що складається з кодів, які містять  $m$  одиниць ( $0 < m < n$ ), мінімальне з них позначимо  $\beta_{min} = \min(B)$ . Цьому коду передуює код  $\beta \geq 2^{n-1}$ , причому  $\beta_{min} = 2\beta - 2^{n-1} + 1$ . Код  $\alpha = \beta - 2^{n-1}$  відрізняється від коду  $\beta$  одиницею в старшому розряд, а отже належить до іншого кільця  $A$ . Оскільки  $\beta_{min} = 2\alpha + 1$ , то у відповідності з наведеною процедурою, на наборі  $X_\alpha$  функція зворотного зв'язку змінить своє значення так, щоб після коду  $\alpha$  слідував код  $\beta_{min}$ . Змінюється також значення функції на коді  $\beta$  так, щоб після цього коду слідував код  $\alpha_{next} = 2\alpha$ . Таким чином, описана вище процедура забезпечує об'єднання кілець  $A$  і  $B$ .

Перехід на мінімальний код кільця  $B$  можливий з одного коду кільця  $A$ :  $L(B) = L(A) + 1$ , за винятком ситуації, коли  $L(B) = 0$ . В останньому випадку один і той самий код є попередником мінімальних кодів двох кілець.

Наприклад, при  $n = 4$  код 1(0001 у двійковому представленні) передуює мініимальному коду 0 кільця  $\{0\}$  та мініимальному коду 1(0001) кільця  $\{1, 2, 4, 8\}$ . Мініимальному коду кожного з кілець (окрім кільця, що складається з нульового коду) передуює код з меншим числом одиниць з іншого кільця. Це означає, що описана процедура забезпечує зв'язування всіх кілець, утворених циклічним зсувом.

### 3.1.2 Приклад роботи алгоритму для $n = 4$

На основі  $n = 4$  розглянемо приклад побудови функції зворотного зв'язку. У відповідності з пунктом 1 обирається початкове значення коду  $j = 8$ . Процес

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 32   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

побудови функції зворотного зв'язку зсувного регістра, що забезпечує максимальний період повторення, приведений в табл. 3.1.

Також для наочнішої візуалізації на рис. 3.1 наведено граф зміни кодів на зсувному регістрі для побудованої функції, де вершинами є усі можливі коди, а дуги графа – відтворює перехід з одного коду в наступний. Розташування окремих вершин на одній лінії не є випадковим – таким чином зображено коди, які розташовуються в одному кільці.

Таблиця 3.1 – Побудова функції зворотнього зв'язку

| Код j | $X_j$ | h  | $u = (2 \cdot j) \bmod 16 + 1$ | R(u)         | F( $X_j$ ) |
|-------|-------|----|--------------------------------|--------------|------------|
| 0     | 0000  | 2  | 1                              | {1,2,4,8}    | 1          |
| 1     | 0001  | 3  | 3                              | {3,6,9,12}   | 1          |
| 2     | 0010  | 13 | 5                              | {5,10}       | 1          |
| 3     | 0011  | 4  | 7                              | {7,14,13,11} | 1          |
| 4     | 0100  | 16 | 9                              | {9,3,6,12}   | 0          |
| 5     | 0101  | 14 | 11                             | {11,7,14,13} | 0          |
| 6     | 0110  | 10 | 13                             | {13,11,7,14} | 0          |
| 7     | 0111  | 5  | 15                             | {15}         | 1          |
| 8     | 1000  | 1  | 1                              | {1,2,4,8}    | 0          |
| 9     | 1001  | 12 | 3                              | {3,6,9,12}   | 0          |
| 10    | 1010  | 15 | 5                              | {5,10}       | 0          |
| 11    | 1011  | 9  | 7                              | {7,14,13,11} | 0          |
| 12    | 1100  | 11 | 9                              | {9,3,6,12}   | 1          |
| 13    | 1101  | 8  | 11                             | {11,7,14,13} | 1          |
| 14    | 1110  | 7  | 13                             | {13,11,7,14} | 1          |
| 15    | 1111  | 6  | 15                             | {15}         | 0          |

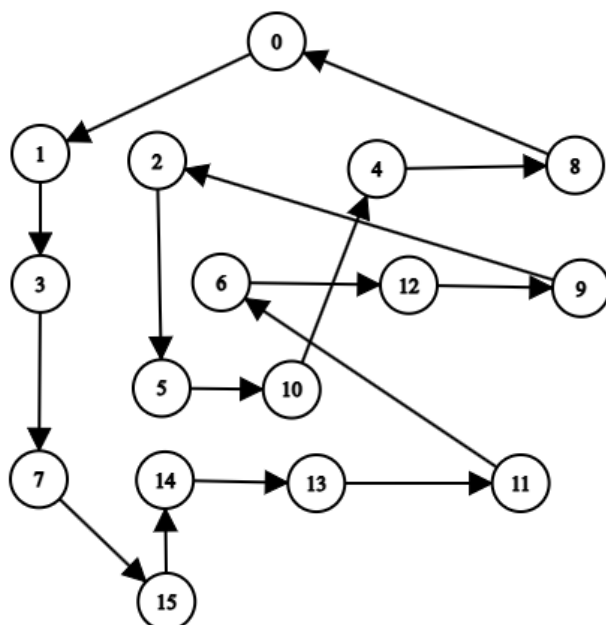


Рисунок 3.1 Граф зміни кодів на зсувному реєстрі

## 3.2 Розширена процедура побудови NLFSR на основі об'єднання мінімальних елементів замкнутих кодових груп

### 3.2.1 Алгоритм синтезу NLFSR

Хоча базова процедура ефективно генерує функцію повного циклу, вона є детермінованою і, по суті, продукує лише одну конкретну функцію для кожної розрядності  $n$ . З точки зору криптографії, наявність широкого спектра унікальних, криптографічно сильних функцій є критично важливою. Чим більше таких функцій може бути згенеровано, тим вищий рівень захисту від атак «з повним перебором за функцією», оскільки простір ключових функцій стає величезним. Тому, розширення базової процедури для отримання множини різних функцій, які також забезпечують повний період, стало наступним логічним кроком.

Розширена процедура надає цю гнучкість, дозволяючи досліднику або розробнику генерувати більше варіацій функцій, що мають бажані властивості повного циклу, підвищуючи їхню унікальність та криптографічну надійність. Це також надає більше простору для вибору функцій з додатковими сприятливими

властивостями, такими як висока нелінійність, балансованість та кореляційна стійкість.

Основна ідея розширення базової процедури полягає в зміні способу з'єднання однієї обраної пари кілець, що виражатиметься у зміні бітів, які відповідають з перехід з одного кільця на інше. З цією метою пропонується об'єднати два довільно обрані циклічні кільця  $A$  і  $B$ , для яких виконуються умови  $L(A) > 0$  та  $L(B) - L(A) = 1$ . Крім того, необхідно заборонити передбачений базовою процедурою перехід на мінімальний код кільця  $B$ .

Розширена процедура об'єднання циклічних кілець складається з виконання наступної послідовності дій:

- 1) Обрати довільний код  $k$ , де  $0 < k < 2^n - 1$ .
- 2) Обрати код  $b \in R(k)$ , який не є мінімальним у циклічному кільці  $R(k)$ :  $b \neq \min(R(k))$ .
- 3) Код  $a$  формується у вигляді:  $a = \left\lfloor \frac{b}{2} \right\rfloor + \varepsilon \cdot 2^{n-1}$ , де  $\lfloor x \rfloor$  - найбільше ціле, менше  $x$ . Значення  $\varepsilon \in \{0, 1\}$  обирається так, щоб  $a \notin R(b)$ .
- 4) Визначити код  $d$ , який передує більшому з двох мінімальних кодів  $R(a)$  і  $R(b)$ :

$$d = \left\lfloor \frac{\max(\min(R(a)), \min(R(b)))}{2} \right\rfloor$$

- 5) Довільно обирається початкове значення поточного коду  $j$ :  $0 < j < 2^n$ . Лічильник  $h$  кодів, для яких визначено значення функції зворотного зв'язку, встановлюється в одиницю:  $h := 1$ .

- 6) Якщо  $j = a$  або  $j = (a + 2^{n-1}) \bmod 2^n$ , то встановити  $f(X_j) = \left\lfloor \frac{j}{2^{n-1}} \right\rfloor \oplus 1 = x_0^j \oplus 1$  і перейти на пункт 9 цієї процедури.

- 7) Якщо  $j = d$  або  $j = (d + 2^{n-1}) \bmod 2^n$ , то встановити  $f(X_j) = \left\lfloor \frac{j}{2^{n-1}} \right\rfloor = x_0^j$  і перейти на пункт 9 цієї процедури.

8) Обчислюється  $u = (2j) \bmod 2^n + 1$ . Якщо обчислений код  $u$  є мінімальним у своєму кільці, тобто  $u = \min(R(u))$ , то значення функції зворотного зв'язку на коді  $j$  визначається як інверсія циклічного зсуву:

$$f(X_j) = \left\lfloor \frac{j}{2^{n-1}} \right\rfloor \oplus 1 = x_0^j \oplus 1,$$

$$\text{а в протилежному випадку } f(X_j) = \left\lfloor \frac{j}{2^{n-1}} \right\rfloor = x_0^j.$$

9) Обчислюється нове значення поточного коду  $j := (2j) \bmod 2^n + f(X_j)$ . Виконується інкремент лічильника:  $h := h + 1$ . Якщо  $h \leq 2^n$ , то здійснюється повернення на п.2, інакше — кінець.

### 3.2.2 Приклад роботи алгоритму для $n = 4$

Проілюструємо розширену процедуру отримання функції зворотного зв'язку для NLFSR з періодом  $2^n$  прикладом при  $n = 4$ .

Згідно з пунктом 1 довільно вибирається код  $k$ :  $0 < k < 15$ . Нехай, наприклад,  $k = 12$ .

Циклічне кільце  $R(12) = \{12(1100), 9(1001), 3(0011), 6(0110)\}$ . Згідно з пунктом 2 код  $b \in R(k)$ ,  $b \neq \min(R(k)) = 3$ . Нехай  $b = 9$ .

Код  $a$  формується як  $a = 4 + \varepsilon \cdot 8$ . Якщо  $\varepsilon = 0$ , то  $a = 4 \notin R(k)$ . Таким чином встановлюємо  $a = 4$ .

Маємо циклічні кільця  $R(12) = \{12, 9, 3, 6\}$  та  $R(4) = \{4, 8, 1, 2\}$ , у яких  $\min(R(b)) = 3$  та  $\min(R(a)) = 1$ . Згідно з пунктом 4 код  $d$  визначається:

$$d = \left\lfloor \frac{\max(3, 1)}{2} \right\rfloor = 1$$

Таким чином,  $a = 4$  і  $a + 2^{n-1} = 12$ ,  $d = 1$  і  $d + 2^{n-1} = 9$ .

Нехай на початку  $j = 0$ . Оскільки

$$j \neq a = 4, j \neq a + 2^{n-1} = 12, j \neq d = 1, j \neq d + 2^{n-1} = 9,$$

то згідно з пунктом 8 обчислюється  $u = 2 \cdot 0 + 1 = 1$ ;  $R(u) = \{1, 2, 4, 8\}$  і  $u = \min(R(u))$ , тому  $f(0000) = 1$  і наступне значення  $j = 1$ .

Оскільки  $j = d = 1$ , то згідно з пунктом 7  $f(X_1) = f(0001) = 0$ . Нове значення  $j$  визначається як  $j = 2 \cdot 1 + 0 = 2$ .

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 36   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Послідовність подальшої побудови функції зворотного зв'язку показана в табл. 3.2. Граф зміни кодів на зсувному реєстрі при використанні побудованої функції зворотного зв'язку показаний на рис. 3.2. Порівнюючи його із рис. 3.1 можна легко візуалізувати принцип роботи розширеного алгоритму – ми змінили з'єднання 1-3 та замінили його на 9-3, а також перехід 9-2 було замінено на 1-2. Змінивши 4 біти сумарно, ми змінили спосіб з'єднання двох кілець

Таблиця 3.2 – Побудова модифікованої функції зворотнього зв'язку

| Код j | $X_j$ | h  | Відношення j до a,d<br>та їм семитричним | $u = (2 \cdot j) \bmod 16 + 1$ | R(u)         | F( $X_j$ ) |
|-------|-------|----|------------------------------------------|--------------------------------|--------------|------------|
| 0     | 0000  | 1  |                                          | 1                              | {1,2,4,8}    | <b>1</b>   |
| 1     | 0001  | 2  | j = d                                    |                                | -            | <b>1</b>   |
| 2     | 0010  | 3  |                                          | 5                              | {5,10}       | <b>1</b>   |
| 3     | 0011  | 8  |                                          | 7                              | {7,14,13,11} | <b>1</b>   |
| 4     | 0100  | 6  | j = a                                    |                                | -            | <b>0</b>   |
| 5     | 0101  | 4  |                                          | 11                             | {11,7,14,13} | <b>0</b>   |
| 6     | 0110  | 14 |                                          | 13                             | {13,11,7,14} | <b>0</b>   |
| 7     | 0111  | 9  |                                          | 15                             | {15}         | <b>1</b>   |
| 8     | 1000  | 16 |                                          | 1                              | {1,2,4,8}    | <b>0</b>   |
| 9     | 1001  | 7  | j = d + 8                                |                                | -            | <b>0</b>   |
| 10    | 1010  | 5  |                                          | 5                              | {5,10}       | <b>0</b>   |
| 11    | 1011  | 13 |                                          | 7                              | {7,14,13,11} | <b>0</b>   |
| 12    | 1100  | 15 | j = a + 8                                |                                | -            | <b>1</b>   |
| 13    | 1101  | 12 |                                          | 11                             | {11,7,14,13} | <b>1</b>   |
| 14    | 1110  | 11 |                                          | 13                             | {13,11,7,14} | <b>1</b>   |
| 15    | 1111  | 10 |                                          | 15                             | {15}         | <b>0</b>   |



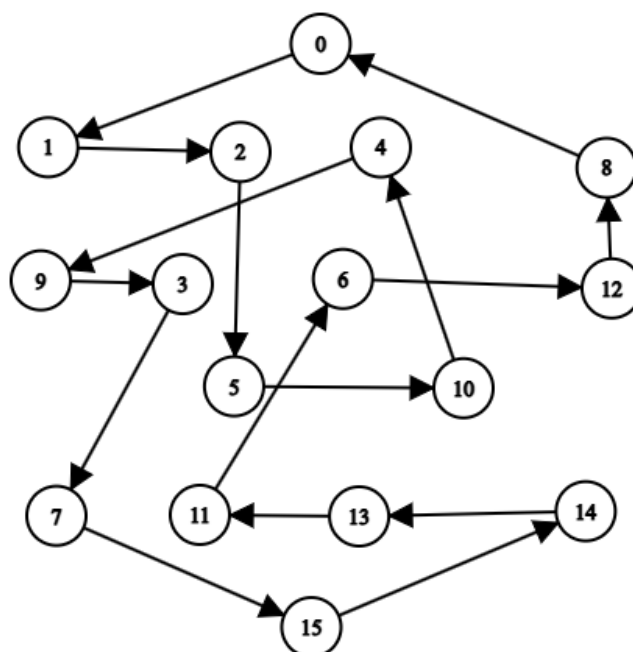


Рисунок 3.2 Граф зміни кодів на зсувному реєстрі

Запропонований метод побудови ГПВП на основі NLFSR забезпечує підвищення ефективності потокових алгоритмів захисту інформації. Використання NLFSR замість LFSR в потокових алгоритмах дозволяє істотно підвищити рівень захищеності даних цим класом алгоритмів. При цьому NLFSR, побудовані з використанням запропонованого методу, забезпечують максимальне значення періоду повторення  $2^n$  та просту апаратну реалізацію.

## ВИСНОВОК ДО РОЗДІЛУ 3

У даному розділі було представлено та детально описано два методи проєктування ГПВДП на NLFSR, спрямовані на забезпечення максимального (повного) періоду роботи генератора, що дорівнює  $2^n$  розрядність регістра.

Перший запропонований метод (базова процедура формування нелінійної функції зворотного зв'язку) ґрунтується на концепції об'єднання циклічних кілець, що утворюються при стандартному циклічному зсуві. Було показано, що шляхом цілеспрямованої модифікації значень функції зворотного зв'язку на кодах, що передують мінімальним елементам кожного кільця, можна об'єднати всі ці кільця в одне загальне кільце, що охоплює всі  $2^n$  станів. Цей метод є фундаментальним і дозволяє отримати одну конкретну нелінійну функцію повного циклу для заданої розрядності. Було наведено приклад побудови такої функції та візуалізацію графу переходів станів.

Другий метод (розширена процедура формування нелінійної функції зворотного зв'язку), є розвитком базового підходу та дозволяє генерувати значно більшу кількість унікальних функцій повного циклу. Суть методу полягає у виборі двох довільних циклічних кілець  $R(a)$  та  $R(b)$ , що задовольняють певним умовам, та модифікації значень функції зворотного зв'язку на ключових кодах  $a$ ,  $b$  та їм симетричним таким чином, щоб забезпечити об'єднання саме цих двох кілець, змінюючи при цьому деякі переходи, передбачені базовою процедурою. Це дає гнучкість у формуванні траєкторії обходу станів і, як наслідок, урізноманітнює можливі результуючі функції повного циклу у порівнянні із базовим методом. Наведено приклад побудови функції за розширеною процедурою та показано її граф.

Розглянуті методи пропонують систематичний підхід до синтезу нелінійних функцій, що забезпечують максимальний період, на відміну від випадкового пошуку чи повного перебору, який є обчислювально неприйнятним для практично значущих  $n$ .

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 39   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Запропоновані методи є теоретичною основою для подальшої програмної реалізації та дослідження властивостей згенерованих послідовностей, що буде розглянуто в наступному розділі. Обидва методи спрямовані на створення функцій, які, завдяки своїй нелінійності та гарантованому повному періоду, мають потенціал для використання в криптографічних застосуваннях.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 40   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

## РОЗДІЛ 4

# РОЗРОБКА ТА АНАЛІЗ ПРОГРАМНИХ ЗАСОБІВ ДЛЯ ПОБУДОВИ ГПВДП НА ЗСУВНИХ РЕГІСТРАХ З НЕЛІНІЙНИМ ЗВОРОТНІМ ЗВ'ЯЗКОМ

### 4.1 Загальна архітектура та структура програмного проєкту

Проєкт було розроблено на платформі Node.js, що забезпечує кросплатформеність та достатню швидкодію для виконання обчислювальних експериментів.



Рисунок 4.1 Node.js

Вибір платформи Node.js для реалізації програмного комплексу був обумовлений кількома ключовими перевагами, які роблять її ідеальною для подібних дослідницьких проєктів. Node.js є кросплатформовою технологією, що дозволяє запускати розроблену систему на різних операційних системах (Windows, macOS, Linux) без необхідності переписування коду, що значно розширює коло потенційних користувачів та розробників. Асинхронна, подієво-орієнтована архітектура Node.js та його висока швидкість виконання JavaScript-коду, особливо для операцій з числовими даними та об'єктами (хоч і без оптимізацій C++ на базовому рівні), забезпечують достатню продуктивність для ефективної симуляції та аналізу алгоритмів на практично значущих, хоча й обмежених, значеннях  $n$ .

Крім того, наявність потужного набору вбудованих модулів, таких як readline для консольної взаємодії, спрощує розробку інтерактивних інтерфейсів

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 41   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

без використання сторонніх залежностей. Модульна архітектура, де кожен компонент (наприклад, cycleChecker, ringUtils, кожен алгоритм) є окремим файлом-модулем, забезпечує високу читабельність коду, спрощує відлагодження, тестування окремих частин та полегшує подальше розширення функціоналу. Це дозволяє командній роботі над проектом та інкрементальному розвитку системи.

Файлова структура розробленого проекту має наступний вигляд:

```
nfsr-generator/
├─ utils/
│   ├── cycleChecker.js      # Модуль перевірки повноти циклу функції
│   └── ringUtils.js         # Допоміжні функції для роботи з циклічними кільцями
├─ algorithms/
│   ├── baseAlgorithm.js     # Реалізація базового алгоритму синтезу функції
│   ├── extendedInteractiveAlgorithm.js # Інтерактивна реалізація розширеного алгоритму
│   └── extendedAutomaticAlgorithm.js  # Автоматична реалізація розширеного алгоритму
└─ index.js                 # Головний файл для взаємодії з користувачем та запуску алгоритмів
```

Рисунок 4.2 Файлова структура проекту

Розглянемо ключові модулі та їхнє призначення:

- 1) utils/cycleChecker.js – містить checkFullCycle(n, feedbackFunctionString), яка приймає розрядність регістра n та функцію зворотного зв'язку у вигляді бітового рядка. Функція симулює роботу зсувного регістра із заданою функцією та перевіряє, чи проходить регістр усі  $2^n$  можливих станів до повторення. Це критично важлива функція для верифікації результатів роботи алгоритмів синтезу.
- 2) utils/ringUtils.js: Надає набір допоміжних функцій для роботи з концепцією циклічних кілець, яка є основою для базового та розширеного методів синтезу. Включає в себе:
  - getCyclicRing(k, n): Повертає масив кодів, що утворюють циклічне кільце при стандартному зсуві, починаючи з коду k для регістра розрядності n.
  - getMinInRing(ring): Знаходить мінімальний код у заданому кільці.
  - decimalToBinaryString(decimal, n): Конвертує десяткове число в двійковий рядок фіксованої довжини n, доповнюючи його нулями зліва за потреби.

3) algorithms/: Папка містить реалізації основних алгоритмів синтезу нелінійних функцій зворотного зв'язку, описаних у Розділі 3. Включає в себе:

- baseAlgorithm.js: Реалізує базовий алгоритм об'єднання циклічних кілець.
- extendedInteractiveAlgorithm.js: Надає користувачеві можливість інтерактивно брати участь у виборі параметрів розширеного алгоритму.
- extendedAutomaticAlgorithm.js: Здійснює автоматичний перебір параметрів для пошуку всіх унікальних функцій повного циклу за допомогою розширеного методу.

4) index.js: Головний скрипт, що забезпечує взаємодію з користувачем через консольне меню. Дозволяє вибрати один з режимів роботи: перевірку функції, запуск базового алгоритму, запуск розширеного інтерактивного алгоритму, запуск розширеного автоматичного алгоритму для пошуку всіх функцій, або вихід з програми.

```
PS C:\Users\ilyha\OneDrive\Markovskiy naukova\Диплом_файли\Code\NLFSR-generator> node .\index.js

--- ГОЛОВНЕ МЕНЮ ---
Виберіть дію:
1. Перевірити повноту циклу для заданої функції
2. Запустити базовий алгоритм
3. Запустити розширений інтерактивний алгоритм
4. Запустити розширений автоматичний алгоритм (пошук усіх унікальних функцій)
5. Вийти з програми
Ваш вибір (1-5): █
```

Рисунок 4.3 Головне меню програми

Обрана архітектура дозволяє легко модифікувати та тестувати кожен компонент окремо, а також забезпечує чітке розмежування логіки.

В основі програмної реалізації лежать три ключові алгоритми: базовий та 2 варіанти розширеного (інтерактивний та автоматичний). Ключовою особливістю програмної реалізації всіх алгоритмів синтезу є використання асинхронних генераторів JavaScript (async function\*). Цей підхід дозволяє перетворити функції обчислення в "поток даних", де за допомогою оператора yield програма може зупинятися на кожному кроці, виводити проміжний результат (стан j, визначене значення f(j)) і потім продовжувати виконання з того самого місця.

Така "покрокова" поведінка використовується для декількох цілей.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 43   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

По-перше, вона забезпечує інтерактивність: користувач може спостерігати за повільним виконанням алгоритму для великих  $n$ , не чекаючи його повного завершення, що для експоненційних алгоритмів може зайняти дуже багато часу.

По-друге, це потужний інструмент для дослідницького аналізу та відлагодження. Деталізований лог кожного переходу, включно з інформацією про те, яким правилом було визначено біт зворотного зв'язку  $f(j)$  — чи то через фіксоване значення (00...0, 11...1), чи через особливі стани розширеного алгоритму, чи за загальним правилом об'єднання кілець — надає цінну інформацію про внутрішню динаміку алгоритму.

Це дозволяє точно верифікувати кожен етап синтезу функції та виявляти неочевидні закономірності у поведінці генератора, що є основою для створення нових методів, в основі яких лежатиме подібна логіка.

## 4.2 Реалізація алгоритмів синтезу NLFSR

### 4.2.1 Реалізація базового алгоритму `baseAlgorithm.js`

Базовий алгоритм призначений для побудови однієї конкретної функції повного циклу для заданої розрядності регістра  $n$ . Ключові етапи роботи програмної реалізації наступні:

1) Ініціалізація:

- Задається розрядність регістра  $n$ .
- Створюється масив `feedbackFunction` розміром  $2^n$ , ініціалізований значеннями -1 (що позначає "не визначено").
- Згідно з теоретичними вимогами, встановлюються фіксовані значення: `feedbackFunction[0] = 1` (для коду 00...0) та `feedbackFunction[numStates - 1] = 0` (для коду 11...1, де  $\text{numStates} = 2^n$ ).
- Поточний код  $j$  ініціалізується початковим значенням (за замовчуванням 0).
- Лічильник кроків (визначених значень функції)  $h$  встановлюється в 0.

2) Основний цикл обходу:

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 44   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

- Цикл виконується, доки не буде пройдено всі  $2^n$  станів або поки поточний код  $j$  не повернеться до початкового значення  $\text{startJ}$ .
- На кожній ітерації для поточного коду  $j$ , якщо значення  $\text{feedbackFunction}[j]$  ще не визначено:
  - Обчислюється код  $u = (2j) \bmod 2^n + 1$ . Це код, який утворюється після зсуву  $j$  і встановлення молодшого розряду в 1 (як кандидат для наступного стану при  $f(j)=1$ ).
  - Знаходиться циклічне кільце  $R(u)$  для коду  $u$  та його мінімальний елемент  $\min(R(u))$ .
  - Якщо  $u = \min(R(u))$ , то значення функції  $f(X_j)$  визначається як інверсія старшого розряду коду  $j$ :  $f(X_j) = x_0^j \oplus 1$ .
  - В іншому випадку,  $f(X_j) = x_0^j$ .
  - Визначене значення записується у  $\text{feedbackFunction}[j]$ .
  - Обчислюється новий поточний код  $j := (2j) \bmod 2^n + \text{feedbackFunction}[j]$ .
  - Лічильник  $h$  інкрементується.
  - Кожен крок, включаючи поточний стан  $j$  (в десятковому та двійковому вигляді), визначене (або попередньо визначене) значення функції зворотного зв'язку для нього, та наступний стан  $j$ , виводяться в консоль за допомогою `yield`.

### 3) Завершення та результат:

- Після завершення циклу обходу формується кінцевий рядок функції зворотного зв'язку `finalFeedbackString` (невизначені значення, якщо такі є, позначаються символом «?»).
- Виводиться отримана функція та загальна кількість кроків  $h$ .
- Видається повідомлення про успішність формування функції повного циклу (якщо  $h = 2^n$  і кінцевий  $j$  збігається з початковим) або про можливу неповність циклу.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 45   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |



Нижче наведено результат роботи програми при обранні базового алгоритму та заданні значення  $n = 4$ :

```
-> Наступний j = 5 (0101)
j = 5 (0101), f = не визначено
-> Наступний j = 10 (1010)
j = 10 (1010), f = не визначено
-> Наступний j = 4 (0100)
j = 4 (0100), f = не визначено
-> Наступний j = 8 (1000)
j = 8 (1000), f = не визначено
-> Наступний j = 0 (0000)

Згенерована функція: 11110000100001110
Довжина шляху: 16
Базовий алгоритм успішно сформував функцію повного циклу.
```

Рисунок 4.4 Результат роботи базового алгоритму

Бачимо, що отримана функція повністю співпадає із визначеною вручну в таблиці 3.1. Це свідчить про коректну роботу алгоритму.

#### 4.2.2 Реалізація розширеного алгоритму в 2 режимах

Розширений алгоритм дозволяє отримати значно більшу кількість функцій повного циклу шляхом зміни об'єднання двох обраних циклічних кілець  $R(a)$  та  $R(b)$ . Він реалізований у двох варіантах: інтерактивному та автоматичному.

Загальна логіка, спільна для обох варіантів розширеного алгоритму:

1) Вибір ключових кодів  $a$ ,  $b$ ,  $d$ :

- Спочатку обирається код  $b$ , який не є мінімальним у своєму циклічному кільці  $R(b)$ .
- Далі, на основі  $b$ , обирається код  $a$  таким чином, щоб кільця  $R(a)$  та  $R(b)$  були різними (перевіряється за нерівністю їх мінімальних елементів).
- Після цього визначається код

$$d = \text{Math.floor}(\max(\min(R(a)), \min(R(b))) / 2).$$

2) Встановлення фіксованих значень функції: Згідно з розширеною процедурою, для кодів  $a$ ,  $d$  та їх симетричних ( $a_{\text{sym}}$ ,  $d_{\text{sym}}$ ) встановлюються фіксовані значення функції зворотного зв'язку. При цьому

важливий порядок: спочатку для  $a$  та  $a_{sym}$ , потім для  $d$  та  $d_{sym}$ , щоб коректно обробити можливі перетини цих кодів.

- $f(a) = X_0^a \oplus 1$
- $f(a_{sym}) = X_0^{a_{sym}} \oplus 1$  (де  $a_{sym} = a \oplus 2^{n-1}$ )
- $f(d) = X_0^d$  (якщо значення ще не встановлено для  $a$  або  $a_{sym}$ )
- $f(d_{sym}) = X_0^{d_{sym}}$  (якщо значення ще не встановлено для  $a$  або  $a_{sym}$ , де  $d_{sym} = d \oplus 2^{n-1}$ )

Основний цикл обходу: Аналогічний до базового алгоритму. Для кожного поточного коду  $j$ , якщо значення  $f(j)$  ще не визначено (тобто  $j$  не є одним з  $a, a_{sym}, d, d_{sym}$ ), воно обчислюється за тим самим правилом, що і в базовому алгоритмі (на основі мінімальності коду  $u = (2j \bmod 2^n) + 1$  у його кільці).

Реалізація інтерактивного варіанту `extendedInteractiveAlgorithm.js`:

1) Взаємодія з користувачем:

- Програма запитує у користувача розрядність регістра  $n$ .
- Потім користувач вводить код  $b$ . Програма перевіряє, чи не є він мінімальним у своєму кільці  $R(b)$ , і за потреби просить ввести інше значення.
- Код  $a$  обирається програмою автоматично з двох можливих варіантів (з  $x_i = 0$  або  $x_i = 1$ ), щоб  $R(a) \neq R(b)$ .
- Код  $d$  також обчислюється автоматично.
- Користувачеві надається можливість обрати початковий код  $j$  для трасування шляху (за замовчуванням береться код 0).

2) Покроковий вивід:

- Кожен крок вибору  $a$ ,  $b$ ,  $d$  та встановлення значень функції для них детально логується.
- Під час основного циклу обходу, для кожного коду  $j$  виводиться інформація про те, як було визначено значення  $f(j)$ : чи було воно встановлене для одного з ключових кодів  $(a, a_{sym}, d, d_{sym})$ , чи було обчислене за стандартним правилом кільця.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 47   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

- Виводиться фінальна функція та аналіз повноти циклу.

Запустимо нашу програму та оберемо цифру «3» для запуску розширеного інтерактивного алгоритму. Оберемо розрядність регістра  $n = 5$ , та візьмемо старндартний початковий код  $j = 0$ . Проміжні результати наведено на рисунку нижче:

```

--- ГОЛОВНЕ МЕНЮ ---
Виберіть дію:
1. Перевірити повноту циклу для заданої функції
2. Запустити базовий алгоритм
3. Запустити розширений інтерактивний алгоритм
4. Запустити розширений автоматичний алгоритм (пошук усіх унікальних функцій)
5. Вийти з програми
Ваш вибір (1-5): 3
Введіть розрядність регістра n: 5

--- Запуск розширеного інтерактивного алгоритму ---
{"step": "Ініціалізація", "n": 5}
Введіть код b ( $0 < b < 32$ ), який не є мінімальним у своєму кільці: 18
{"step": "Код b обрано", "b": 18, "ringOfB": [18, 5, 10, 20, 9], "minRingOfB": 5}
{"step": "Код a обрано", "a": 25, "xi": 1, "ringOfA": [25, 19, 7, 14, 28], "minRingOfA": 7}
{"step": "Встановлено f для a та симетричного a", "a_decimal": 25, "a_binary": "11001", "f_a": 0, "symmetricA_decimal": 9, "symmetricA_binary": "01001", "f_sym_a": 1}
{"step": "Код d визначено", "d": 3, "symmetricD": 19}
{"step": "Встановлено f для d та симетричного d", "d_decimal": 3, "d_binary": "00011", "f_d": 0, "symmetricD_decimal": 19, "symmetricD_binary": "10011", "f_sym_d": 1}
Бажаєте ввести інший початковий код j для обходу (стандартно 0)? (0 - ні, 1 - так): 0
{"step": "Початок обходу", "startJForPath": 0, "current_j_binary": "00000"}
j = 0 (00000), f = 1 (обчислено за правилом кільця)

```

Рисунок 4.5 Проміжні результати роботи розширеного інтерактивного алгоритму

Після цього програма починає вираховувати значення функції на кожному кроці (а також перевіряє рівність коду  $j$  із  $a, a_{sym}, d, d_{sym}$ ), і виводить в консоль знайдену функцію повного циклу:

```

j = 28 (11100), f = 1 (обчислено за правилом кільця)
-> Наступний j = 25 (11001)
j = 25 (11001), f = 0 (визначено для a)
-> Наступний j = 18 (10010)
j = 18 (10010), f = 0 (обчислено за правилом кільця)
-> Наступний j = 4 (00100)
j = 4 (00100), f = 0 (обчислено за правилом кільця)
-> Наступний j = 8 (01000)
j = 8 (01000), f = 0 (обчислено за правилом кільця)
-> Наступний j = 16 (10000)
j = 16 (10000), f = 0 (обчислено за правилом кільця)
-> Наступний j = 0 (00000)

Згенерована функція: 11100101010000010001101010111110
Довжина шляху: 32
Розширений інтерактивний алгоритм успішно сформував функцію.

```

Рисунок 4.6 Результат роботи розширеного інтерактивного алгоритму

Реалізація автоматичного варіанту `extendedAutomaticAlgorithm.js`:

1) Перебір параметрів:

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 48   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

- Програма ітерує по всіх можливих значеннях коду  $b$  (від 1 до  $2^n - 1$ ), ретельно перевіряючи, що  $b$  не є мінімальним у своєму кільці  $R(b)$ . Це ключовий етап для забезпечення різноманітності результуючих функцій.
- Для кожного такого  $b$ , програма перебирає обидва варіанти  $x_i \in \{0,1\}$  для формування коду  $a$ .
- Якщо  $R(a) \neq R(b)$  (що верифікується за нерівністю їх мінімальних елементів), обчислюється  $d$ , який є критично важливим для задання "точки злиття" кілець у розширеному методі.

## 2) Побудова та перевірка функції:

- Для кожної валідної комбінації параметрів ( $b$ ,  $a$ ,  $d$ ) генерується повна таблиця функції зворотного зв'язку, використовуючи логіку встановлення фіксованих значень та обходу решти станів (як в інтерактивному режимі, але без участі користувача).
- Після формування повної функції, невизначені значення (якщо такі залишились) заповнюються за правилом симетрії  $f(s_{sym}) = f(s) \oplus 1$  або за замовчуванням  $f(s) = s_0$ .
- Використовується функція `checkFullCycle` для верифікації того, що згенерована функція дійсно забезпечує повний цикл  $2^n$ .

## 3) Забезпечення унікальності:

- Для коректного обліку та уникнення дублювання результатів, програма використовує об'єкт `Set (uniqueFunctions)`. Перед тим, як додати нову успішно згенеровану функцію до переліку знайдених, її бітовий рядок конвертується у строкове представлення і перевіряється на наявність у `Set`. Це забезпечує, що в кінцевому виводі будуть лише унікальні функції повного циклу.

## 4) Вивід результатів:

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 49   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

- По мірі знаходження кожної нової унікальної функції, в консоль виводиться її порядковий номер, параметри (b, a, d, хіщо були використані для її генерації, та сама функція у вигляді бітового рядка. Після завершення обробки всіх можливих комбінацій параметрів, програма підсумовує свою роботу, виводячи загальну кількість знайдених унікальних функцій повного циклу. Це дозволяє досліднику оцінити розмаїття функцій, доступних для заданої розрядності n за допомогою даного розширеного методу.

Цей режим призначений для автоматичного пошуку та генерації всіх можливих унікальних функцій повного циклу за допомогою розширеного методу для заданого n без участі користувача у виборі елементів.

Результат роботи програми із використанням розширеного автоматичного алгоритму наведено на рисунку 4.7.

```

5. Вийти з програми
Ваш вибір (1-5): 4
Введіть розрядність регістра n: 4

--- Запуск розширеного автоматичного алгоритму ---

ЗНАЙДЕНО УНІКАЛЬНУ ФУНКЦІЮ №1:
Параметри генерації: k_початковий=2, b=2, xi=1, a=9, d=1
Функція: 1111000100001110

ЗНАЙДЕНО УНІКАЛЬНУ ФУНКЦІЮ №2:
Параметри генерації: k_початковий=8, b=8, xi=1, a=12, d=1
Функція: 1011100101000110

ЗНАЙДЕНО УНІКАЛЬНУ ФУНКЦІЮ №3:
Параметри генерації: k_початковий=10, b=10, xi=1, a=13, d=3
Функція: 1110010100011010

ЗНАЙДЕНО УНІКАЛЬНУ ФУНКЦІЮ №4:
Параметри генерації: k_початковий=12, b=12, xi=1, a=14, d=3
Функція: 1110001100011100

Розширений автоматичний алгоритм завершив роботу.
Всього знайдено унікальних функцій повного циклу: 4

```

Рисунок 4.7 Результат роботи розширеного автоматичного алгоритму

### 4.3 Аналіз отриманих результатів та обчислювальної складності

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 50   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Розроблений програмний комплекс дозволяє не лише синтезувати нелінійні функції зворотного зв'язку, але й здійснювати глибокий аналіз самого процесу їх побудови. Використання генераторів JavaScript для реалізації алгоритмів дало змогу отримувати цінну інформацію про послідовність обходу станів та ухвалення рішень на кожному кроці, що є надзвичайно важливим для всебічного розуміння динаміки цих детермінованих систем. Це дозволяє не тільки верифікувати функціональність, але й виявляти неочевидні властивості та взаємозв'язки.

#### Аналіз складності:

- 1) Базовий алгоритм: теоретична складність визначається кількістю станів, які необхідно обійти ( $2^n$ ), помноженою на складність операцій на кожному кроці. Ключовими операціями є визначення значення функції зворотного зв'язку, що включає обчислення  $u$ , знаходження кільця  $R(u)$  та його мінімуму. Генерація кільця може вимагати до  $n$  зсувів для кожного з його елементів, а кількість кілець буде близько  $\frac{2^n}{n}$ . Таким чином, складність операцій з кільцями може бути порядку  $O(2^n)$  у найгіршому випадку. Тому складність алгоритму буде орієнтовно  $O(2^{2n})$ .
- 2) Розширений інтерактивний алгоритм: обчислювальна складність однієї ітерації, після того як користувач ввів параметри  $b$  (та автоматично визначилися  $a$  та  $d$ ), аналогічна базовому алгоритму, оскільки основна частина — це обхід станів. Таким чином, його складність також є порядку  $O(2^{2n})$  для одного сеансу генерації.
- 3) Розширений автоматичний алгоритм: є найбільш обчислювально затратним. Він включає зовнішній цикл перебору для  $b$  (майже  $2^n$  варіантів), внутрішній для  $x_i$  (2 варіанти), і для кожної вдалої комбінації параметрів виконується повний обхід, аналогічно до розширеного інтерактивного. Це призводить до загальної складності, що прямує до  $O(2^{3n+1})$ , що робить його практично застосовним лише для дуже малих значень  $n$ .

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 51   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Можливість покрокового логування, реалізована за допомогою генераторів, виявилася надзвичайно корисною для відлагодження та розуміння того, як саме встановлюються значення функції зворотного зв'язку на різних етапах алгоритму, що дозволяє проводити глибинний аналіз поведінки NLFSR, подібно до сучасних методів бісимуляції [27].

Кількість функцій NLFSR для деяких значень  $n$ , які отримано в результаті роботи розширеного автоматичного алгоритму, наведено у таблиці 4.1.

Таблиця 4.1 – Кількість функцій NLFSR для деяких значень  $n$

|                      |   |    |    |    |    |     |     |     |      |
|----------------------|---|----|----|----|----|-----|-----|-----|------|
| Довжина регістру $n$ | 4 | 5  | 6  | 7  | 8  | 9   | 10  | 11  | 12   |
| Кількість функцій    | 4 | 10 | 20 | 46 | 94 | 198 | 406 | 838 | 1698 |

Основне обмеження цього алгоритму – це його висока обчислювальна складність, що робить неможливим пряме застосування автоматичного алгоритму для великих  $n$  ( $n > 100$  для реальних сучасних криптографічних систем). Це обмежує його використання до теоретичних досліджень та невеликих демонстраційних прикладів..

Також користувач завжди може перевірити, чи буде функція, задана у вигляді бітового рядка символів «0» та «1», генерувати повний цикл генератора для заданого значення  $n$ .

При  $n=5$  перевіримо дві функції:

- 1)  $F1 = 1111010100000001000010101111110$  – функція, що синтезована з базового алгоритму при  $n = 5$ ;
- 2)  $F2 = 1111010100000001000010101111111$  – функція  $F1$ , у якої останній біт змінений на 1, через що функція вже не буде генерувати повний цикл (не буде виходу зі стану 11...1).

Переконаємось, що програма коректно обробляє обидві функції:

```

--- ГОЛОВНЕ МЕНЮ ---
Виберіть дію:
1. Перевірити повноту циклу для заданої функції
2. Запустити базовий алгоритм
3. Запустити розширений інтерактивний алгоритм
4. Запустити розширений автоматичний алгоритм (пошук усіх унікальних ф
ункцій)
5. Вийти з програми
Ваш вибір (1-5): 1
Введіть розрядність регістра n: 5
Введіть функцію зворотного зв'язку для n=5 (бітовий рядок довжини 32):
1111010100000001000010101111110
Функція ЗАБЕЗПЕЧУЄ повний цикл.

```

Рисунок 4.8 Результат перевірки F1

```

--- ГОЛОВНЕ МЕНЮ ---
Виберіть дію:
1. Перевірити повноту циклу для заданої функції
2. Запустити базовий алгоритм
3. Запустити розширений інтерактивний алгоритм
4. Запустити розширений автоматичний алгоритм (пошук усіх унікальних ф
ункцій)
5. Вийти з програми
Ваш вибір (1-5): 1
Введіть розрядність регістра n: 5
Введіть функцію зворотного зв'язку для n=5 (бітовий рядок довжини 32):
1111010100000001000010101111111
Функція НЕ ЗАБЕЗПЕЧУЄ повний цикл.

```

Рисунок 4.9 Результат перевірки F2

Результати, які проілюстровані на рисунках 4.8 та 4.9, повністю підтверджують функціональну коректність та надійність розробленого програмного комплексу. Це демонструє, що реалізовані алгоритми точно відтворюють логіку синтезу функцій зворотного зв'язку згідно з теоретичними положеннями, а модуль перевірки циклу ефективно верифікує властивості генерованих послідовностей.



## ВИСНОВОК ДО РОЗДІЛУ 4

У даному розділі було детально описано програмну реалізацію базового та розширених (інтерактивного та автоматичного) алгоритмів синтезу нелінійних функцій зворотного зв'язку для зсувних регістрів, призначених для генерації псевдовипадкових двійкових послідовностей повного циклу. Цей етап роботи був критично важливим для перетворення теоретичних моделей у дієвий інструментарій, що дозволяє не тільки генерувати ці функції, але й покроково візуалізувати процес їх побудови завдяки використанню асинхронних генераторів JavaScript. Це забезпечило глибоке розуміння внутрішніх механізмів функціонування розроблених методів.

Було представлено структуру проєкту, висвітлено призначення кожного ключового модуля та їх взаємодію, що демонструє продуманий підхід до архітектури програмного рішення. Особлива увага була приділена аналізу логіки роботи кожного з реалізованих режимів програми, включаючи інтерактивну взаємодію з користувачем, яка дозволяє вручну керувати деякими параметрами синтезу, та режим автоматичного пошуку унікальних функцій. Останній, незважаючи на свою значну обчислювальну складність, довів свою здатність ефективно знаходити множину унікальних функцій повного циклу для невеликих значень розрядності  $n$ , що повністю відповідає теоретичним засадам розширеного методу об'єднання циклічних кілець, описаного в попередніх розділах. Таким чином, програмний комплекс успішно слугує засобом верифікації та демонстрації теоретичних концепцій.

Окрім основних алгоритмів синтезу, у ході розробки та тестування програмного комплексу було створено та використано низку допоміжних функцій. До них, зокрема, відносяться функції для точної перевірки повноти циклу (які виявилися незамінними для валідації генерованих функцій) та функції для роботи з циклічними кільцями (визначення елементів кільця, пошук мінімального елемента). Ці інструменти, реалізовані як окремі модулі, не тільки

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 54   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

дозволили ретельно верифікувати результати роботи основних алгоритмів, але й сприяли виявленню певних, раніше неочевидних закономірностей у структурі функцій повного циклу та самому процесі їх формування. Зібрані спостереження, аналіз проміжних станів, а також розроблений програмний інструментарій є не просто результатом реалізації, а цінним надбанням, що може лягти в основу подальших досліджень.

Зокрема, отримані дані відкривають перспективи для глибшого аналізу зв'язків між структурою циклічних кілець та впливом вибору параметрів об'єднання на властивості результуючих нелінійних функцій. Такий аналіз може дозволити перейти від перебірних до більш детермінованих або евристичних методів вибору оптимальних параметрів, що, в свою чергу, є ключем до розробки більш складних, ефективних та швидших алгоритмів синтезу NLFSR для практично значущих великих значень розрядності регістра  $n$ . Це підвищить прикладну цінність розробленої системи у сфері криптографії та інформаційної безпеки.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 55   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

# ВИСНОВКИ

У дипломному проєкті було розглянуто актуальну проблему розробки методів та програмних засобів для побудови ГПВДП, що задовольняють високим вимогам криптографічної стійкості та ефективності. Основну увагу було зосереджено на генераторах на основі нелінійних регістрів зсуву як перспективному напрямку для посилення безпеки сучасних інформаційних систем.

У першому розділі було проведено всебічний огляд існуючих підходів до генерації псевдовипадкових послідовностей. Проаналізовано переваги та недоліки різних класів генераторів, від класичних лінійних до більш сучасних, заснованих на потокових та блокових шифрах. Це дозволило обґрунтувати необхідність дослідження саме нелінійних генераторів для досягнення високої криптографічної міцності.

Другий розділ присвячено теоретичному дослідженню властивостей нелінійних функцій зворотного зв'язку, які є ключовими для забезпечення максимального періоду роботи генераторів. Було сформульовано та доведено необхідні умови для таких функцій, що значно звужує простір їхнього пошуку, та проаналізовано кількість потенційно придатних нелінійних рішень, що вказує на їхню перевагу над лінійними аналогами.

У третьому розділі розроблено два нових методи синтезу нелінійних функцій зворотного зв'язку для NLFSR: базову процедуру на основі детермінованого об'єднання циклічних кілець та розширену процедуру, що дозволяє генерувати більшу кількість унікальних функцій повного циклу на основі базової процедури із певною модифікацією. Ці методи пропонують систематичний підхід до конструювання ефективних нелінійних генераторів.

Четвертий розділ описує програмну реалізацію запропонованих методів на платформі Node.js. Створений програмний комплекс включає інструменти для перевірки повноти циклу та реалізацію базового і розширеного алгоритмів

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 56   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

синтезу з можливістю покрокового логування та автоматичного пошуку унікальних функцій. Проведено аналіз отриманих результатів та обчислювальної складності.

Загалом, у дипломному проєкті успішно поєднано теоретичні дослідження з практичною розробкою, результатом чого стали методи синтезу криптографічно значущих нелінійних ГПВДП та програмний інструментарій для їх побудови й аналізу. Це створює міцне підґрунтя для подальших досліджень у напрямку розробки ще більш ефективних алгоритмів для великих розрядностей. Корисність роботи полягає у наданні інструментів та методів для створення генераторів з покращеною криптографічною стійкістю, що є важливим для захисту інформації в сучасних цифрових системах.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 57   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

## СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Menezes A.J. Handbook of Applied Cryptography / A.J. Menezes, P.C. van Oorschot, S.A. Vanstone. – Boca Raton: CRC Press. – 1996. – 780 p.
2. Eastlake D. Randomness Requirements for Security / D. Eastlake, J. Schiller, S. Crocker. – RFC 4086. – 2005. – 51 p.
3. Лисенко І.М. Криптографія. Основи та засоби захисту інформації / І.М. Лисенко, А.О. Сизоненко. – Київ: КНУ. – 2015. – 216 с.
4. Мороз В.І. Інформаційна безпека: теорія і практика захисту інформації / В.І. Мороз, В.С. Шевченко. – Харків: ХНУРЕ, 2017. – 288 с.
5. Golomb S.W. Shift Register Sequences. Revised edition / S.W. Golomb. – Washington, D.C.: Aegean Park Press. – 1982. – 247 p.
6. Shannon C.E. Communication Theory of Secrecy Systems / C.E. Shannon // The Bell System Technical Journal. – 1949. – Vol. 28, № 4. – P. 656–715.
7. Yao A.C. Theory and Applications of Trapdoor Functions / A.C. Yao // Proc. 23rd Annual Symposium on Foundations of Computer Science. – 1982. – P. 80–91.
8. Kolmogorov A.N. Three Approaches to the Quantitative Definition of Information / A.N. Kolmogorov // Problems of Information Transmission. – 1965. – Vol. 1, № 1. – P. 1–7.
9. Knuth D.E. The Art of Computer Programming. Volume 2: Seminumerical Algorithms / D.E. Knuth. – Boston: Addison-Wesley. – 1997. – 762 p.
10. Lehmer D.H. Mathematical Methods in Large-Scale Computing Units / D.H. Lehmer // Proc. of a Second Symposium on Large-Scale Digital Calculating Machinery. – 1949. – P. 141–146.
11. Lewis H.R. A Note on Polynomial Congruential Pseudorandom Number Generators / H.R. Lewis, R.K. Payne // BIT. – 1984. – Vol. 24, № 2. – P. 211–213.
12. Rivest R.L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems / R.L. Rivest, A. Shamir, L. Adleman // Communications of the ACM. – 1978. – Vol. 21, № 2. – P. 120–126.

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 58   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

13. Hegazy A. Improving Digital Signaling on Time Varying Channels using RAKE Receiver / A. Hegazy, A. Zekry. – PhD thesis. – 2013. – 156 p.
14. Gutierrez J. A. On the Linear and Nonlinear Complexity Profile on Nonlinear Pseudorandom Number Generators / Gutierrez J., Shparlinski I., A., Winterhof A. // IEEE Trans. Information Theory. – 2003. – Vol. 49, № 1. – P. 60-64.
15. Дудикевич В.Б. Дослідження параметрів модифікованого генератора Джіффі / В.Б. Дудикевич, В.М. Максимович, Ю.М. Костів // Матеріали І міжнародної науково-практичної конференції "Безпека та захист інформації в інформаційних і телекомунікаційних системах". – Харків: ХНЕУ. – 2008. – № 6. – С. 74–76.
16. Feng Y. A Novel Nonlinear Pseudorandom Sequence Generator for the Fractal Function / Y. Feng, H. Wang, C. Chang, H. Lu, F. Yang, C. Wang // Fractal and Fractional. – 2022. – Vol. 6, № 10. – P. 589. DOI: 10.3390/fractalfract6100589.
17. Al-Hinai S. Algebraic Attacks on Clock-Controlled Stream Ciphers / S. Al-Hinai, L. Batten, B. Colbert, K. Wong // Lecture Notes in Computer Science. – 2006. – Vol. 4058. – P. 1–16. DOI: 10.1007/11780656\_1.
18. Bikos A. Random Number Generators: Principles and Applications / A. Bikos, P.E. Nastou, G. Petroudis, Y. C. Stamatiou // Cryptography. – 2023. – Vol. 7, № 4. – P. 54. – DOI: 10.3390/cryptography7040054.
19. Болгарський А.В. Інформаційно-математичні основи комп'ютерної безпеки: підручник / А.В. Болгарський, А.Н. Власюк, А.В. Кабанов. – Київ: Університет «Україна». – 2005. – 248 с.
20. Fredricksen H. A Survey of Full-Length Shift Register Sequences / H. Fredricksen // SIAM Review. – 1982. – Vol. 24, № 2. – P. 195–221.
21. Марковський О.П. Ефективний метод побудови нелінійних генераторів для телекомунікаційних систем / О.П. Марковський, Зохре Карім Заде Сейфоллах, В. Є. Гурін // Електроника и связь. Тематический выпуск «Проблемы электроники» ч.3. – ПЦ «Аверс» – 2007. – С. 87-89.
22. Марковський О.П. Побудова генераторів псевдовипадкових послідовностей на зсувному регістрі з нелінійним зворотним зв'язком / О.П. Марковский, Хан

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.003 ПЗ | Арк. |
|     |      |          |        |      |                    | 59   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

Йонг Ли, Л. Б. Ермоленко // Вісник Національного технічного університету України "КПІ". Інформатика, управління та обчислювальна техніка. 2004, – № 41. – С. 75-84.

- 23.Марковський О.П. Метод побудови ефективного нелінійного генератора псевдовипадкових двійкових послідовностей / О. П. Марковський, Л. Б. Ермоленко // Вісник Національного технічного університету України "КПІ". Інформатика, управління та обчислювальна техніка. 2003, - № 39. - С.15-23.
- 24.Cusick T.W. Cryptographic Boolean Functions and Applications / T.W. Cusick, P. Stanica. – San Diego: Academic Press. – 2009. – 513 p.
25. Carlet C. Boolean Functions for Cryptography and Error Correcting Codes / C. Carlet // Lecture Notes in Computer Science. – Berlin: Springer. – 2010. – Vol. 1494. – P. 257–397.
- 26.Zhong J. On minimum period of nonlinear feedback shift registers on grain-like structures / J.Zhong, D.Lin // IEEE Transaction on Information Theory. – 2018. – Vol. 64, № 9. – P.6429-6442. DOI: 10.1109/TIT.18.
- 27.Zhe Gao. Bisimulations of nonlinear feedback shift registers / Gao Zhe, Feng Jun // International Journal of Control. – 2025. – Vol.98. – № 1. – P.208-219. DOI: 10.1080/00207179.2024.2329728.

# **ДОДАТОК 1**

**Метод та програмні засоби побудови генераторів псевдовипадкових двійкових  
послідовностей для криптографічних застосувань**

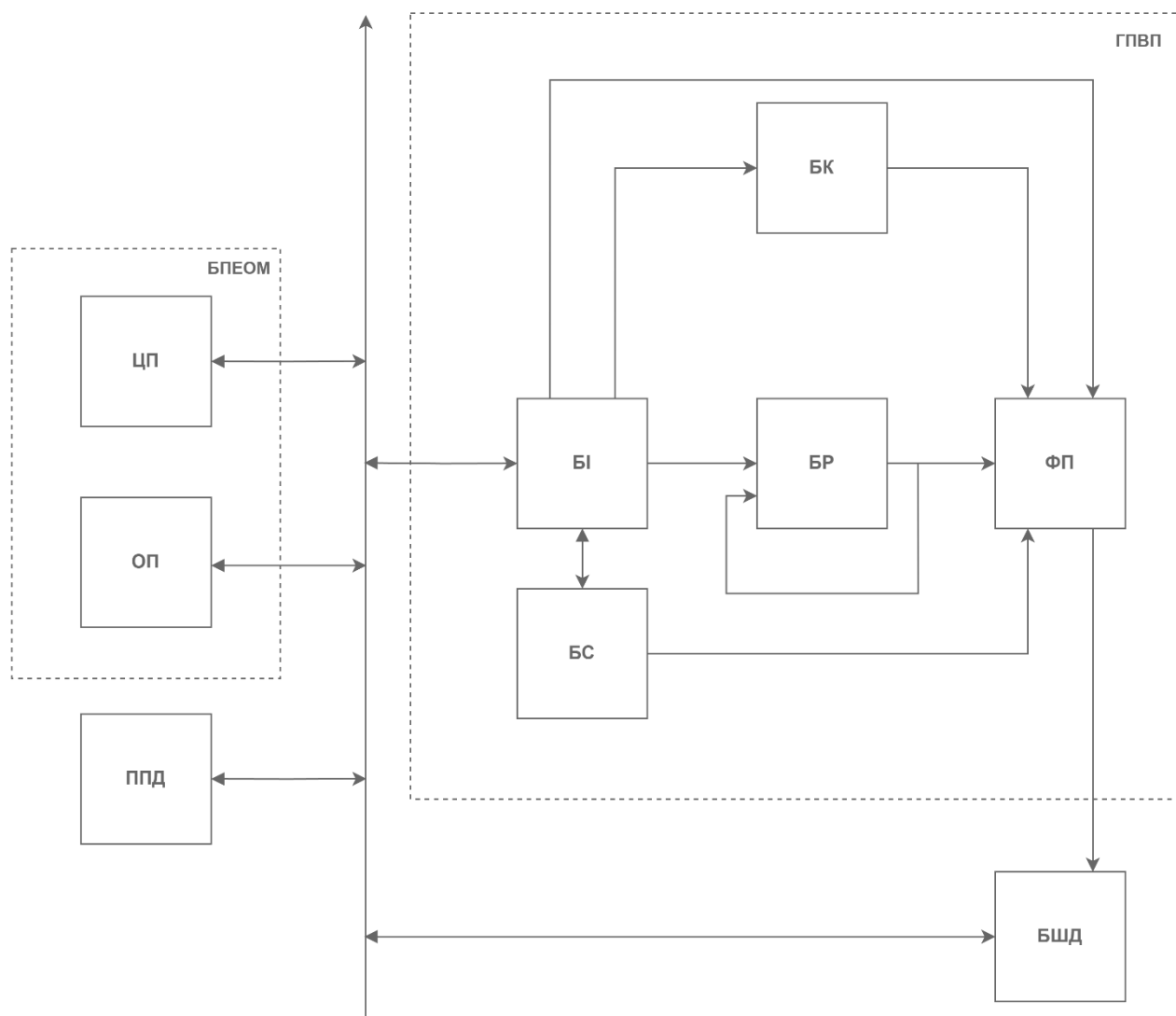
**Структура програмно-апаратного комплексу для  
генерації ПВП (структурна схема)**

**ІАЛЦ.467200.004 Д1**

**Аркушів 1**

**Київ 2025 р**





БІ – блок інтерфейсу  
БПЕОМ – базова персональна електронно-обчислювальна машина  
БР – блок регістрів  
БС – блок синхронізації  
БК – блок керування  
БШД – блок шифрування/дешифрування

ГПВП – генератор псевдовипадкових послідовностей  
ОП – основна пам'ять  
СШ – системна шина  
ППД – пристрій передачі даних  
ФП – функціональний перетворювач  
ЦП – центральний процесор

|           |                 |          |        |      |                                                                                                                                                                                                               |                                           |       |         |
|-----------|-----------------|----------|--------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------|---------|
|           |                 |          |        |      | ІАЛЦ.467200.004 Д1                                                                                                                                                                                            |                                           |       |         |
|           |                 | № докум. | Підпис | Дата |                                                                                                                                                                                                               |                                           |       |         |
| Розробив  | Наливайко І. І. |          |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань<br><br>Структура програмно-апаратного комплексу для генерації ПВП (структурна схема) | Літ.                                      | Аркуш | Аркушів |
| Перевірив | Марковський О.  |          |        |      |                                                                                                                                                                                                               |                                           | 1     | 1       |
| Реценз.   | Дичка І. А.     |          |        |      |                                                                                                                                                                                                               | КПІ ім. Ігоря Сікорського,<br>ФІОТ, ІМ-11 |       |         |
| Н. Контр. | Мищенко Л. Д.   |          |        |      |                                                                                                                                                                                                               |                                           |       |         |
| Затвердив |                 |          |        |      |                                                                                                                                                                                                               |                                           |       |         |

## **ДОДАТОК 2**

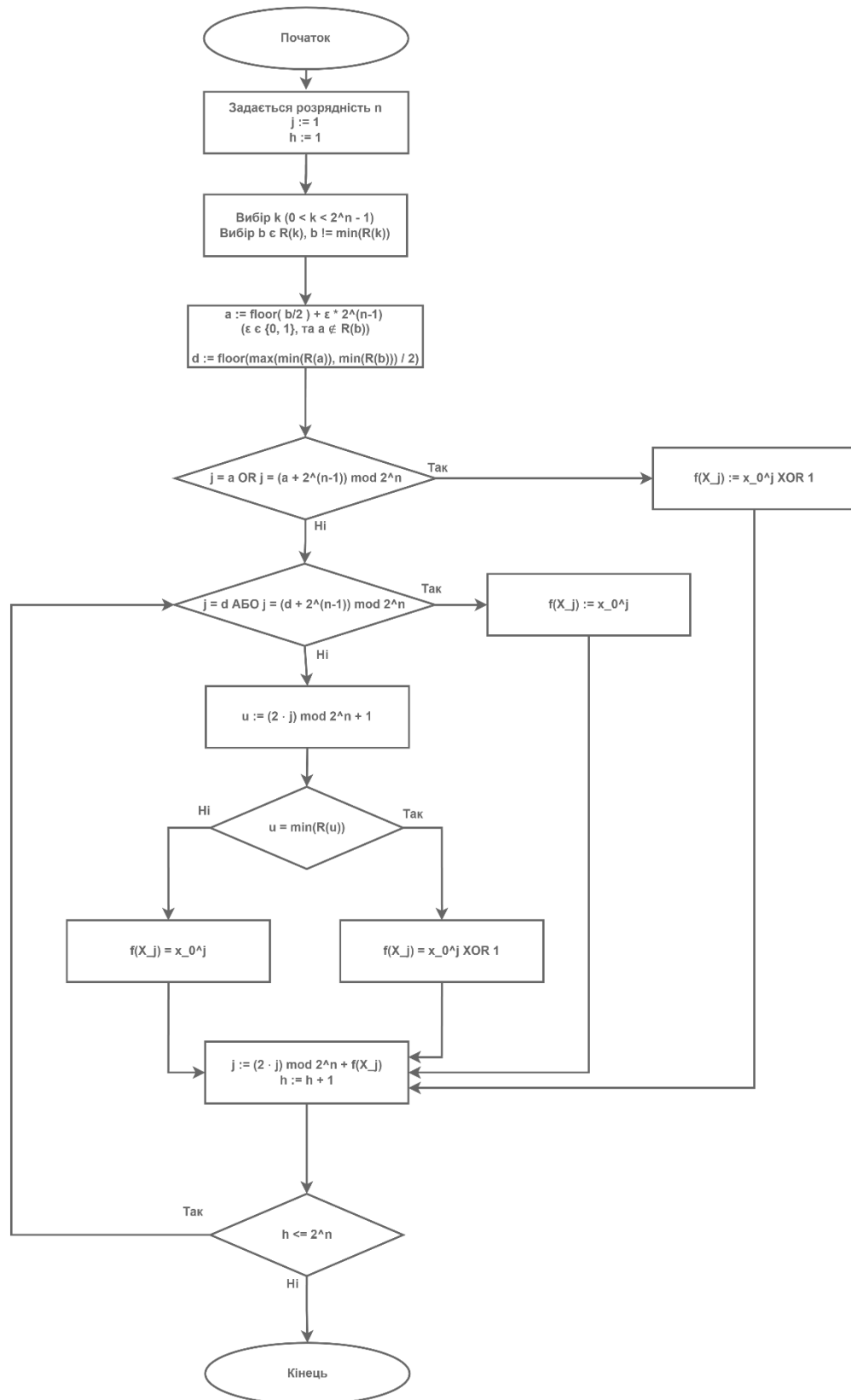
Метод та програмні засоби побудови генераторів псевдовипадкових двійкових  
послідовностей для криптографічних застосувань

**Блок-схема розширеного алгоритму**

ІАЛЦ.467200.005 Д2

Аркушів 1

Київ 2025 р



|           |                   |          |        |      |                                                                                                                                                                                              |                                           |       |         |
|-----------|-------------------|----------|--------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------|---------|
|           |                   |          |        |      | ІАЛЦ.467200.005 Д2                                                                                                                                                                           |                                           |       |         |
|           |                   | № докум. | Підпис | Дата | Метод та програмні засоби побудови генераторів<br>псевдовипадкових двійкових послідовностей для<br>криптографічних застосувань<br><br>Блок-схема розширеного алгоритму<br>(принципова схема) | Літ.                                      | Аркуш | Аркушів |
| Розробив  | Наливайко І. І.   |          |        |      |                                                                                                                                                                                              |                                           | 1     | 1       |
| Перевірив | Марковський О. П. |          |        |      |                                                                                                                                                                                              | КПІ ім. Ігоря Сікорського,<br>ФІОТ, ІМ-11 |       |         |
| Реценз.   | Дичка І. А.       |          |        |      |                                                                                                                                                                                              |                                           |       |         |
| Н. Контр. | Міщенко Л. Д.     |          |        |      |                                                                                                                                                                                              |                                           |       |         |
| Затвердив |                   |          |        |      |                                                                                                                                                                                              |                                           |       |         |

## **ДОДАТОК 3**

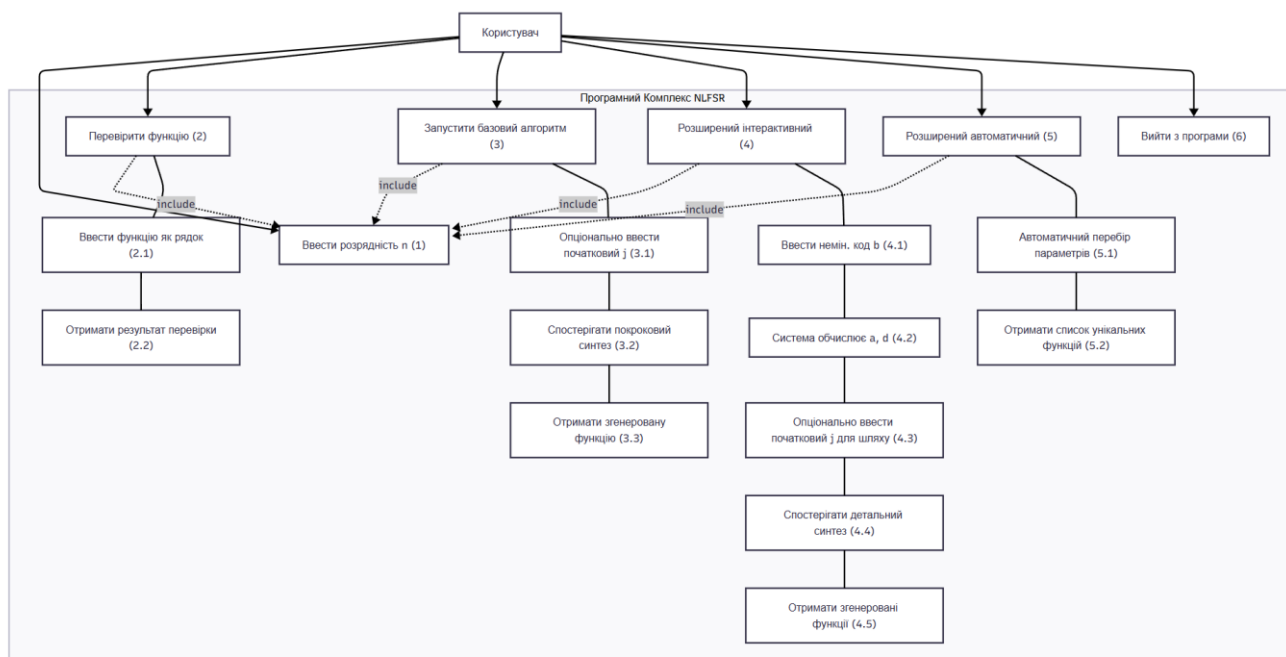
Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань

**Діаграма функціональних вимог (функціональна схема)**

**ІАЛЦ.467200.006 ДЗ**

Аркушів 1

**Київ 2025 р**



|           |                   |          |        |      |                                                                                                                                                                                              |                                           |       |         |
|-----------|-------------------|----------|--------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------|---------|
|           |                   |          |        |      | ІАЛЦ.467200.006 ДЗ                                                                                                                                                                           |                                           |       |         |
|           |                   | № докум. | Підпис | Дата | Метод та програмні засоби побудови генераторів<br>псевдовипадкових двійкових послідовностей для<br>криптографічних застосувань<br><br>Діаграма функціональних вимог<br>(функціональна схема) | Літ.                                      | Аркуш | Аркушів |
| Розробив  | Наливайко І. І.   |          |        |      |                                                                                                                                                                                              |                                           | 1     | 1       |
| Перевірив | Марковський О. П. |          |        |      |                                                                                                                                                                                              | КПІ ім. Ігоря Сікорського,<br>ФІОТ, ІМ-11 |       |         |
| Реценз.   | Дичка І. А.       |          |        |      |                                                                                                                                                                                              |                                           |       |         |
| Н. Контр. | Мищенко Л. Д.     |          |        |      |                                                                                                                                                                                              |                                           |       |         |
| Затвердив |                   |          |        |      |                                                                                                                                                                                              |                                           |       |         |

## **ДОДАТОК 4**

Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань

**Текст програмного коду**

**ІАЛЦ.467200.007 Д4**

Аркушів 12

**Київ 2025 р**

**cycleChecker.js:**

```
function checkFullCycle(n, feedbackFunctionString) {
  const numStates = 1 << n; // 2^n
  if (feedbackFunctionString.length !== numStates) {
    throw new Error(`Довжина функції зворотного зв'язку (${feedbackFunctionString.length})
не відповідає 2^n (${numStates}) для n=${n}`);
  }

  const feedbackFunction = feedbackFunctionString.split("").map(Number);
  const visitedStates = new Array(numStates).fill(false);
  let currentState = 0; // Початковий стан - всі нулі
  let count = 0;

  for (let i = 0; i < numStates; i++) {
    if (visitedStates[currentState]) {
      return false;
    }
    visitedStates[currentState] = true;
    count++;

    const feedbackBit = feedbackFunction[currentState];
    currentState = ((currentState << 1) & (numStates - 1)) | feedbackBit;
  }
  return count === numStates;
}

module.exports = { checkFullCycle };
```

**ringUtils.js:**

```
function getCyclicRing(k, n) {
  const numStates = 1 << n;
  const ring = [];
  let currentState = k;
  do {
    ring.push(currentState);
    const старшийБіт = (currentState >> (n - 1)) & 1;
    currentState = ((currentState << 1) & (numStates - 1)) | старшийБіт;
  } while (currentState !== k && ring.length < numStates);
  return ring;
}
```

|           |                   |          |        |      |                                                                                                                                                        |                                               |       |         |
|-----------|-------------------|----------|--------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|-------|---------|
|           |                   |          |        |      | ІАЛЦ.467200.007 Д4                                                                                                                                     |                                               |       |         |
|           |                   | № докум. | Підпис | Дата |                                                                                                                                                        |                                               |       |         |
| Розробив  | Наливайко І. І.   |          |        |      | Метод та програмні засоби побудови генераторів псевдовипадкових двійкових послідовностей для криптографічних застосувань<br><br>Текст програмного коду | Літ.                                          | Аркуш | Аркушів |
| Перевірив | Марковський О. П. |          |        |      |                                                                                                                                                        |                                               | 1     | 12      |
| Реценз.   | Дичка І. А.       |          |        |      |                                                                                                                                                        | КПІ ім. Ігоря Сікорського,<br><br>ФІОТ, ІМ-11 |       |         |
| Н. Контр. | Міщенко Л. Д.     |          |        |      |                                                                                                                                                        |                                               |       |         |
| Затвердив |                   |          |        |      |                                                                                                                                                        |                                               |       |         |

```
function getMinInRing(ring) {
  return Math.min(...ring);
}
```

```
function decimalToBinaryString(decimal, n) {
  return decimal.toString(2).padStart(n, '0');
}
```

```
module.exports = { getCyclicRing, getMinInRing, decimalToBinaryString };
```

### baseAlgorithm.js:

```
const { getCyclicRing, getMinInRing, decimalToBinaryString } = require('../utils/ringUtils');

async function* baseAlgorithmGenerator(n, startJ = 0) {
  const numStates = 1 << n;
  const feedbackFunction = new Array(numStates).fill(-1); // -1 означає "не визначено"

  feedbackFunction[0] = 1; // f(0...0) = 1
  const allOnesState = numStates - 1;
  feedbackFunction[allOnesState] = 0; // f(1...1) = 0

  // Встановлюємо початковий код j
  let j = startJ;
  let h = 0;

  yield { step: 'Ініціалізація', j, feedbackFunction: [...feedbackFunction] };

  do {
    const binaryJ = decimalToBinaryString(j, n);
    yield {
      type: 'крок',
      j_decimal: j,
      j_binary: binaryJ,
      current_feedback_value: feedbackFunction[j] !== -1 ? feedbackFunction[j] : 'не визначено'
    };

    if (feedbackFunction[j] === -1) {
      // Обчислюємо  $u = (2*j) \bmod 2^n + 1$ 
      const u_candidate_after_shift_without_feedback = (j << 1) & (numStates - 1);
      const u = u_candidate_after_shift_without_feedback | 1;

      const ringOfU = getCyclicRing(u, n);
      const minRingOfU = getMinInRing(ringOfU);

      let fj;
      if (u === minRingOfU) {
        // значення функції інверсії циклічного зсуву

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 2    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |



```

        const x0j = (j >> (n - 1)) & 1;
        fj = x0j ^ 1;
    } else {
        const x0j = (j >> (n - 1)) & 1;
        fj = x0j;
    }
    feedbackFunction[j] = fj;
}

// Обчислюємо нове значення j
j = ((j << 1) & (numStates - 1)) | feedbackFunction[j];
h++;

yield { type: 'новий_j', j_decimal: j, j_binary: decimalToBinaryString(j,n) };

} while (h < numStates && j !== startJ);

const finalFeedbackString = feedbackFunction.map(bit => bit === -1 ? '?' : bit).join("");
yield { type: 'результат', feedbackFunctionString: finalFeedbackString, path_length: h };

if (h === numStates && j === startJ) {
    yield { type: 'завершення', message: 'Базовий алгоритм успішно сформував функцію повного циклу.', complete: true };
} else {
    yield { type: 'завершення', message: `Базовий алгоритм завершився. Довжина шляху: ${h}. Кінцевий стан j: ${j}. Функція може бути не повного циклу.`, complete: false };
}
}

module.exports = { baseAlgorithmGenerator };

```

#### **extendedInteractiveAlgorithm.js:**

```

const readline = require('readline').createInterface({
    input: process.stdin,
    output: process.stdout,
    terminal: false
});

const { getCyclicRing, getMinInRing, decimalToBinaryString } = require('../utils/ringUtils');

function askQuestion(query) {
    return new Promise(resolve => readline.question(query, ans => {
        resolve(parseInt(ans));
    }));
}

async function* extendedInteractiveAlgorithmGenerator(n) {
    const numStates = 1 << n;
    const feedbackFunction = new Array(numStates).fill(-1);

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 3    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

```

yield { step: 'Ініціалізація', n };

// Крок 2: Одразу обираємо немінімальний код b
let b;
let ringOfB, minRingOfB;
let isValidB = false;
while(!isValidB) {
  const candidateB = await askQuestion(`Введіть код b ( $0 < b < \{\text{numStates}\}$ ), який не є
мінімальним у своєму кільці: `);
  if (candidateB <= 0 || candidateB >= numStates) {
    console.log("Некоректний діапазон для b.");
    continue;
  }
  ringOfB = getCyclicRing(candidateB, n);
  minRingOfB = getMinInRing(ringOfB);
  if (candidateB !== minRingOfB) {
    b = candidateB;
    isValidB = true;
  } else {
    console.log(`Код  $\{\text{candidateB}\}$  є мінімальним у своєму кільці  $\{\text{ringOfB.join(', ')}\}$ .
Будь ласка, оберіть інший.`);
  }
}
yield { step: 'Код b обрано', b, ringOfB, minRingOfB };

//Код a формується
let a;
let xi; // 0 або 1
let foundA = false;
for (let currentXi = 0; currentXi <= 1; currentXi++) {
  a = Math.floor(b / 2) + currentXi * (1 << (n - 1));
  const ringOfA_candidate = getCyclicRing(a,n);
  if (getMinInRing(ringOfA_candidate) !== minRingOfB) { // R(a) != R(b)
    xi = currentXi;
    foundA = true;
    break;
  }
}

if (!foundA) {
  yield { type: 'помилка', message: `Не вдалося знайти відповідний код a для b= $\{\text{b}\}$ .` };
  readline.close();
  return;
}

const ringOfA = getCyclicRing(a,n);
const minRingOfA = getMinInRing(ringOfA);
yield { step: 'Код a обрано', a, xi, ringOfA, minRingOfA };

feedbackFunction[a] = ((a >> (n - 1)) & 1) ^ 1;
const symmetricA = a ^ (1 << (n - 1));

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 4    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

```

    if (symmetricA >= 0 && symmetricA < numStates) {
        feedbackFunction[symmetricA] = ((symmetricA >> (n-1)) & 1) ^ 1;
    }
    yield { step: 'Встановлено f для a та симетричного a', a_decimal: a, a_binary:
decimalToBinaryString(a, n), f_a: feedbackFunction[a],
        symmetricA_decimal: symmetricA, symmetricA_binary:
decimalToBinaryString(symmetricA, n), f_sym_a: feedbackFunction[symmetricA] };

const d = Math.floor(Math.max(minRingOfA, minRingOfB) / 2);
const symmetricD = d ^ (1 << (n - 1));
yield { step: 'Код d визначено', d, symmetricD };

if(feedbackFunction[d] === -1) {
    feedbackFunction[d] = (d >> (n - 1)) & 1;
}
if (symmetricD >= 0 && symmetricD < numStates && feedbackFunction[symmetricD] === -1)
{
    feedbackFunction[symmetricD] = (symmetricD >> (n - 1)) & 1;
}
yield { step: 'Встановлено f для d та симетричного d', d_decimal: d, d_binary:
decimalToBinaryString(d, n), f_d: feedbackFunction[d],
        symmetricD_decimal: symmetricD, symmetricD_binary:
decimalToBinaryString(symmetricD, n), f_sym_d: feedbackFunction[symmetricD] };

//Довільно вибирається початкове значення j для обходу
let currentJ = 0;
if (await askQuestion('Бажаєте ввести інший початковий код j для обходу (стандартно 0)? (0
- ні, 1 - так): ') === 1) {
    currentJ = await askQuestion('Введіть початковий код j для обходу (0 <= j <
${numStates}): ');
}

let h = 0;
const startJForPath = currentJ;

yield { step: 'Початок обходу', startJForPath: currentJ, current_j_binary:
decimalToBinaryString(currentJ, n) };

do {
    const binaryJ_current = decimalToBinaryString(currentJ, n);
    let logMessage = `j = ${currentJ} (${binaryJ_current}), `;

    let fj_current;
    // Перевіряємо, чи є поточний j одним з "особливих" кодів
    if (currentJ === a) {
        fj_current = feedbackFunction[a];
        logMessage += `f = ${fj_current} (визначено для a)`;
    } else if (currentJ === symmetricA) {

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 5    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

```

    fj_current = feedbackFunction[symmetricA];
    logMessage += `f = ${fj_current} (визначено для симетричного a)`;
  } else if (currentJ === d) {
    fj_current = feedbackFunction[d];
    logMessage += `f = ${fj_current} (визначено для d)`;
  } else if (currentJ === symmetricD) {
    fj_current = feedbackFunction[symmetricD];
    logMessage += `f = ${fj_current} (визначено для симетричного d)`;
  } else if (feedbackFunction[currentJ] !== -1) { // Якщо вже визначено раніше
    fj_current = feedbackFunction[currentJ];
    logMessage += `f = ${fj_current} (вже визначено)`;
  }
  else {
    const u_candidate_after_shift_without_feedback = (currentJ << 1) & (numStates - 1);
    const u = u_candidate_after_shift_without_feedback | 1;

    const ringOfU = getCyclicRing(u, n);
    const minRingOfU = getMinInRing(ringOfU);

    if (u === minRingOfU) {
      const x0_current_j = (currentJ >> (n - 1)) & 1;
      fj_current = x0_current_j ^ 1;
    } else {
      const x0_current_j = (currentJ >> (n - 1)) & 1;
      fj_current = x0_current_j;
    }
    feedbackFunction[currentJ] = fj_current;
    logMessage += `f = ${fj_current} (обчислено за правилом кільця)`;
  }
  yield { type: 'крок', message: logMessage };

  currentJ = ((currentJ << 1) & (numStates - 1)) | fj_current;
  h++;

  yield { type: 'новий_j', j_decimal: currentJ, j_binary: decimalToBinaryString(currentJ, n) };

  } while (h < numStates && currentJ !== startJForPath);

  const finalFeedbackString = feedbackFunction.map(bit => bit === -1 ? '?' : bit).join(""); // ? для
невстановлених
  yield { type: 'результат', feedbackFunctionString: finalFeedbackString, path_length: h };

  if (h === numStates && currentJ === startJForPath) {
    yield { type: 'завершення', message: 'Розширений інтерактивний алгоритм успішно
сформував функцію.', complete: true };
  } else {
    yield { type: 'завершення', message: `Розширений інтерактивний алгоритм завершився.
Довжина шляху: ${h}. Кінцевий стан j: ${currentJ}. Функція може бути не повного циклу.`,
complete: false };
  }
}

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 6    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

```

    readline.close();
}

module.exports = { extendedInteractiveAlgorithmGenerator };

extendedAutomaticAlgorithm.js:

const { getCyclicRing, getMinInRing, decimalToBinaryString } = require('../utils/ringUtils');
const { checkFullCycle } = require('../utils/cycleChecker');

async function* extendedAutomaticAlgorithmGenerator(n) {
    const numStates = 1 << n;
    let functionsFoundCount = 0;
    const uniqueFunctions = new Set();

    yield { type: 'старт_авто', n };

    // Перебір усіх можливих b, які НЕ є мінімальними у своїх кільцях
    for (let candidateB = 1; candidateB < numStates; candidateB++) {
        const ringOfB = getCyclicRing(candidateB, n);
        const minRingOfB = getMinInRing(ringOfB);

        if (candidateB === minRingOfB) continue;

        const b = candidateB;

        // Перебір обох варіантів xi для формування a
        for (let xi = 0; xi <= 1; xi++) {
            const feedbackFunction = new Array(numStates).fill(-1);

            let a = Math.floor(b / 2) + xi * (1 << (n - 1));
            const ringOfA = getCyclicRing(a, n);
            const minRingOfA = getMinInRing(ringOfA);

            if (minRingOfA === minRingOfB) continue; // Кільця R(a) та R(b) повинні бути різними

            // Встановлення значень функції для a та симетричного йому
            feedbackFunction[a] = ((a >> (n-1)) & 1) ^ 1;
            const symmetricA = a ^ (1 << (n-1));
            if (symmetricA >= 0 && symmetricA < numStates) {
                feedbackFunction[symmetricA] = ((symmetricA >> (n-1)) & 1) ^ 1;
            }

            //Визначення d
            const d = Math.floor(Math.max(minRingOfA, minRingOfB) / 2);
            const symmetricD = d ^ (1 << (n - 1));

            // Встановлення значень функції для d та симетричного йому
            if(feedbackFunction[d] === -1) {
                feedbackFunction[d] = (d >> (n - 1)) & 1;
            }
        }
    }
}

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 7    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

```

    }
    if (symmetricD >= 0 && symmetricD < numStates && feedbackFunction[symmetricD]
=== -1) {
        feedbackFunction[symmetricD] = (symmetricD >> (n-1)) & 1;
    }

    // Подальший обхід для формування решти значень
    let currentJ = 0; // Завжди починаємо з 0 для уніфікації
    let h = 0;
    const visitedInPath = new Array(numStates).fill(false);

    let pathIsFullCycle = true;
    let generatedPathLogForThisFunc = "";

    for(let pathStep = 0; pathStep < numStates; pathStep++) {
        if (visitedInPath[currentJ]) {
            pathIsFullCycle = false;
            break;
        }
        visitedInPath[currentJ] = true;

        const binaryJ_current_auto = decimalToBinaryString(currentJ, n);
        let logMessageAuto = `j = ${currentJ} (${binaryJ_current_auto}), `;
        let fj_current_auto;

        if (currentJ === a) {
            fj_current_auto = feedbackFunction[a];
            logMessageAuto += `f = ${fj_current_auto} (для a)`;
        } else if (currentJ === symmetricA) {
            fj_current_auto = feedbackFunction[symmetricA];
            logMessageAuto += `f = ${fj_current_auto} (для sym_a)`;
        } else if (currentJ === d) {
            fj_current_auto = feedbackFunction[d];
            logMessageAuto += `f = ${fj_current_auto} (для d)`;
        } else if (currentJ === symmetricD) {
            fj_current_auto = feedbackFunction[symmetricD];
            logMessageAuto += `f = ${fj_current_auto} (для sym_d)`;
        } else if (feedbackFunction[currentJ] !== -1) {
            fj_current_auto = feedbackFunction[currentJ];
            logMessageAuto += `f = ${fj_current_auto} (вже визначено)`;
        } else {
            const u_candidate_auto = (currentJ << 1) & (numStates - 1);
            const u_auto = u_candidate_auto | 1;
            const ringOfU_auto = getCyclicRing(u_auto, n);
            const minRingOfU_auto = getMinInRing(ringOfU_auto);

            if (u_auto === minRingOfU_auto) {
                fj_current_auto = ((currentJ >> (n - 1)) & 1) ^ 1;
            } else {
                fj_current_auto = (currentJ >> (n - 1)) & 1;
            }
        }
    }

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 8    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

```

        feedbackFunction[currentJ] = fj_current_auto;
        logMessageAuto += `f = ${fj_current_auto} (за правилом кільця)`;
    }
    generatedPathLogForThisFunc += logMessageAuto + ` -> Наступний j: `;
    currentJ = ((currentJ << 1) & (numStates - 1)) | fj_current_auto;
    generatedPathLogForThisFunc +=
`$ {currentJ} ($ {decimalToBinaryString(currentJ,n)})\n`;
    h++;
}

if (pathIsFullCycle && h === numStates && currentJ === 0) {
    for (let state = 0; state < numStates; state++) {
        if (feedbackFunction[state] === -1) {
            const symmetricState = state ^ (1 << (n-1));
            if (feedbackFunction[symmetricState] !== -1) {
                feedbackFunction[state] = feedbackFunction[symmetricState] ^ 1;
            } else {
                feedbackFunction[state] = (state >> (n-1)) & 1;
            }
        }
    }
}

const finalFeedbackString = feedbackFunction.join("");

if (!uniqueFunctions.has(finalFeedbackString)) {
    if (checkFullCycle(n, finalFeedbackString)) {
        uniqueFunctions.add(finalFeedbackString);
        functionsFoundCount++;
        yield {
            type: 'знайдено_унікальну_функцію',
            count: functionsFoundCount,
            params: { k: candidateB, initial_b: b, xi, a, d },
            feedbackFunctionString: finalFeedbackString,
        };
    }
}

yield { type: 'завершення_авто', totalUniqueFunctionsFound: functionsFoundCount, message:
`Розширений автоматичний алгоритм завершив роботу.` };
}

module.exports = { extendedAutomaticAlgorithmGenerator };

```

#### index.js:

```

const readline = require('readline').createInterface({
    input: process.stdin,
    output: process.stdout,
    terminal: false

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 9    |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |

```

});
const { checkFullCycle } = require('./utils/cycleChecker');
const { baseAlgorithmGenerator } = require('./algorithms/baseAlgorithm');
const { extendedInteractiveAlgorithmGenerator } =
require('./algorithms/extendedInteractiveAlgorithm');
const { extendedAutomaticAlgorithmGenerator } =
require('./algorithms/extendedAutomaticAlgorithm');

async function main() {
  let running = true;

  while (running) {
    console.log("\n--- ГОЛОВНЕ МЕНЮ ---");
    console.log("Виберіть дію:");
    console.log("1. Перевірити повноту циклу для заданої функції");
    console.log("2. Запустити базовий алгоритм");
    console.log("3. Запустити розширений інтерактивний алгоритм");
    console.log("4. Запустити розширений автоматичний алгоритм (пошук усіх унікальних функцій)");
    console.log("5. Вийти з програми");

    const choiceStr = await askQuestion("Ваш вибір (1-5): ");
    const choice = parseInt(choiceStr);

    if (isNaN(choice) || choice < 1 || choice > 5) {
      console.error("Некоректний вибір.");
      continue;
    }

    if (choice === 5) {
      console.log("Завершення роботи програми...");
      running = false;
      readline.close();
      return;
    }

    const nStr = await askQuestion("Введіть розрядність регістра n: ");
    const n = parseInt(nStr);

    if (isNaN(n) || n <= 0) {
      console.error("Некоректне значення n.");
      continue;
    }

    switch (choice) {
      case 1:
        const funcString = await askQuestion(`Введіть функцію зворотного зв'язку для n=${n}
(бітовий рядок довжини ${1 << n}): `);
        try {
          const isFull = checkFullCycle(n, funcString);

```

|     |      |          |        |      |                    |      |
|-----|------|----------|--------|------|--------------------|------|
|     |      |          |        |      | ІАЛЦ.467200.007 Д4 | Арк. |
|     |      |          |        |      |                    | 10   |
| Зм. | Арк. | № докум. | Підпис | Дата |                    |      |



```

        console.log(' Функція ${isFull ? 'ЗАБЕЗПЕЧУЄ' : 'НЕ ЗАБЕЗПЕЧУЄ'} повний
цикл. ');
    } catch (e) {
        console.error("Помилка:", e.message);
    }
    break;
case 2:
    console.log("\n--- Запуск базового алгоритму ---");
    let startJ_base = 0;
    const useCustomStartJStr = await askQuestion("Використати інший початковий код j
для базового алгоритму? (0 - ні (стандартно 0), 1 - так): ");
    if (parseInt(useCustomStartJStr) === 1) {
        const startJ_base_str = await askQuestion("Введіть початковий код j: ");
        startJ_base = parseInt(startJ_base_str);
    }

    for await (const result of baseAlgorithmGenerator(n, startJ_base)) {
        if (result.type === 'крок') {
            console.log(`j = ${result.j_decimal} (${result.j_binary}), f =
${result.current_feedback_value}`);
        } else if (result.type === 'новий_j') {
            console.log(` -> Наступний j = ${result.j_decimal} (${result.j_binary})`);
        } else if (result.type === 'результат') {
            console.log(`\nЗгенерована функція: ${result.feedbackFunctionString}`);
            console.log(`Довжина шляху: ${result.path_length}`);
        } else if (result.type === 'завершення'){
            console.log(result.message);
        } else {
            console.log(JSON.stringify(result));
        }
    }
    break;
case 3:
    console.log("\n--- Запуск розширеного інтерактивного алгоритму ---");
    for await (const result of extendedInteractiveAlgorithmGenerator(n)) {
        if (result.type === 'крок') {
            console.log(result.message);
        } else if (result.type === 'новий_j') {
            console.log(` -> Наступний j = ${result.j_decimal} (${result.j_binary})`);
        } else if (result.type === 'результат') {
            console.log(`\nЗгенерована функція: ${result.feedbackFunctionString}`);
            console.log(`Довжина шляху: ${result.path_length}`);
        } else if (result.type === 'завершення'){
            console.log(result.message);
        } else if (result.type === 'помилка') {
            console.error("ПОМИЛКА:", result.message);
            break;
        }
        else {
            console.log(JSON.stringify(result));
        }
    }
}

```

```

        break;
    case 4:
        console.log("\n--- Запуск розширеного автоматичного алгоритму ---");
        for await (const result of extendedAutomaticAlgorithmGenerator(n)) {
            if (result.type === 'знайдено_унікальну_функцію') {
                console.log(`\nЗНАЙДЕНО УНІКАЛЬНУ ФУНКЦІЮ №${result.count}:`);
                console.log(`  Параметри генерації: k_початковий=${result.params.k},
b=${result.params.initial_b}, xi=${result.params.xi}, a=${result.params.a},
d=${result.params.d}`);
                console.log(`  Функція: ${result.feedbackFunctionString}`);
            } else if (result.type === 'завершення_авто') {
                console.log(`\n${result.message}`);
                console.log(`Всього знайдено унікальних функцій повного циклу:
${result.totalUniqueFunctionsFound}`);
            } else if (result.type === 'старт_авто' || result.type === 'не_повний_цикл') {
                // console.log(JSON.stringify(result));
            }
        }
        break;
    default:
        console.log("Невірний вибір.");
    }
}

function askQuestion(query) {
    return new Promise(resolve => readline.question(query, ans => {
        resolve(ans);
    }));
}

main();

```